

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

École Supérieure en Informatique

-08 Mai 1945- Sidi Bel Abbès



Final Year Thesis

Combined Master 2 and Engineering Degree

French Title

Conception et Déploiement d'un Jumeau Numérique pour l'Optimisation
d'un Système d'Irrigation Intelligent en Agriculture de Précision

English Title

Design and Deployment of a Digital Twin for the Optimization of an
Intelligent Irrigation System in Precision Agriculture

Presented by:

Lebaili Mohamed Ouail – Computer Systems Engineering (CIS)

Senhadji Mohamed Said – Artificial Intelligence and Data Science (AIDS)

Supervised by: [Insert supervisor names]

Academic year: 2025–2026

Dedication

To our families, teachers, and everyone who supported us throughout this final-year project. This work is dedicated to those who believe that engineering can serve sustainability, agriculture, and society.

Acknowledgements

We would like to express our sincere gratitude to our supervisors, teachers, and the staff of École Supérieure en Informatique -08 Mai 1945- Sidi Bel Abbès for their guidance, scientific advice, and support during the realization of this project. We also thank the laboratory team who made the smart farming prototype, Libelium Smart Agriculture kit, XBee communication infrastructure, Raspberry Pi gateway, and experimental environment available for the implementation and validation of this work.

This thesis also benefited from recent research on Digital Twins, smart irrigation, precision agriculture, Internet of Things, machine learning, and sustainable agricultural water management. These works allowed us to position the proposed platform not only as a software prototype, but also as a research-driven engineering contribution.

Abstract

Agriculture is increasingly affected by water scarcity, climate variability, soil degradation, and the growing demand for food production. Irrigation management is one of the most critical factors in this context, because traditional irrigation practices are often based on fixed schedules, manual observation, or empirical experience. Such approaches can lead to water waste, plant stress, and reduced productivity. In response, this thesis presents the design and deployment of Agrio, a Digital Twin-based intelligent irrigation platform for precision agriculture.

The proposed system integrates a physical smart farming prototype, Libelium Smart Agriculture sensing equipment, XBee radio-frequency communication, a Raspberry Pi gateway, HiveMQ/MQTT messaging, a FastAPI backend, a PostgreSQL database, a Random Forest-based machine learning service, a Digital Twin state and simulation layer, a Next.js web dashboard, a Flutter mobile application, a WeMos actuator, and a water pump. The platform receives real sensor data from a controlled prototype environment, normalizes and stores the measurements, updates the virtual state of the irrigation zone, predicts irrigation need, simulates future moisture evolution, generates recommendations, supports human validation, schedules irrigation actions, and controls the physical pump through MQTT commands.

The Master 2 contribution of this work is the scientific study of Digital Twins in agriculture, with a focus on their maturity levels, enabling technologies, irrigation applications, and limitations. The engineering contribution is the implementation of an end-to-end cyber-physical prototype that connects sensing, communication, backend processing, artificial intelligence, simulation, user interfaces, deployment, and actuation. The results show that the system goes beyond a classical IoT monitoring dashboard by providing synchronization, prediction, what-if simulation, recommendation, scheduling, and physical control. The current implementation remains a prototype tested in a controlled environment, and future work should extend it to long-term field validation, stronger agronomic calibration, cybersecurity, and advanced optimization techniques.

Keywords: Digital Twin, precision agriculture, smart irrigation, IoT, MQTT, XBee, Raspberry Pi, FastAPI, PostgreSQL, machine learning, Random Forest, water management, WeMos, smart farming.

Résumé

L'agriculture moderne fait face à des défis importants liés à la rareté de l'eau, aux variations climatiques et à la nécessité d'optimiser la production agricole. Dans ce contexte, l'irrigation intelligente constitue une application centrale de l'agriculture de précision. Ce mémoire présente la conception et le déploiement d'Agrio, une plateforme d'irrigation intelligente basée sur le concept de jumeau numérique. Le système proposé combine un prototype physique, un kit Libelium Smart Agriculture, une communication radiofréquence XBee, une passerelle Raspberry Pi, la messagerie MQTT/HiveMQ, un backend FastAPI, une base de données PostgreSQL, un service d'apprentissage automatique basé sur Random Forest, une couche de simulation du jumeau numérique, un tableau de bord web, une application mobile Flutter, un actionneur WeMos et une pompe d'eau.

La contribution scientifique porte sur l'étude de l'état de l'art des jumeaux numériques en agriculture et leur positionnement par rapport aux systèmes d'irrigation intelligents existants. La contribution d'ingénierie réside dans la mise en œuvre d'une chaîne complète reliant les capteurs, la communication IoT, le traitement backend, la prédiction, la simulation, la recommandation, la validation humaine et le contrôle physique de l'irrigation. Les résultats montrent que le système dépasse la simple visualisation IoT en intégrant la synchronisation, la prédiction, la simulation et l'action.

Mots-clés : jumeau numérique, agriculture de précision, irrigation intelligente, IoT, MQTT, apprentissage automatique, FastAPI, PostgreSQL.

ملخص

تواجه الزراعة الحديثة تحديات كبيرة مرتبطة بندرة المياه والتغيرات المناخية والحاجة إلى تحسين الإنتاجية. في هذا السياق، تمثل أنظمة الري الذكي أحد أهم تطبيقات الزراعة الدقيقة. يقدم هذا العمل منصة **Agrio** وهي منصة ري ذكية تعتمد على مفهوم التوأم الرقمي، حيث تجمع بين نموذج فيزيائي حقيقي، حساسات زراعية، اتصال لاسلكي، بوابة **Pi، Raspberry** بروتوكول **MQTT**، خادم خلفي **FastAPI** قاعدة بيانات **PostgreSQL** خدمة تعلم آلي، محاكاة رقمية، واجهة ويب، تطبيق هاتف، ومتحكم **WeMos** للتحكم في مضخة الماء.

تتمثل المساهمة العلمية في دراسة حالة الفن للتوائم الرقمية في الزراعة ومقارنة الأعمال الحالية، بينما تتمثل المساهمة الهندسية في بناء سلسلة كاملة من جمع البيانات إلى التنبيه والمحاكاة والتوصية والتحكم الفعلي في المضخة. يبين العمل أن النظام لا يقتصر على مراقبة البيانات، بل يوفر تمثيلاً رقمياً متزامناً ودعماً ذكياً لاتخاذ القرار.

List of Abbreviations

Table 1: Abbreviations used in the thesis

Abbreviation	Meaning
AI	Artificial Intelligence
ADT	Agricultural Digital Twin
API	Application Programming Interface
CIS	Computer Systems Engineering
AIDS	Artificial Intelligence and Data Science
CPS	Cyber-Physical System
DB	Database
DSS	Decision Support System
DT	Digital Twin
ET	Evapotranspiration
ETo	Reference Evapotranspiration
FAO	Food and Agriculture Organization
FMIS	Farm Management Information System
IoT	Internet of Things
JSON	JavaScript Object Notation
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
REST	Representational State Transfer
RF	Radio Frequency
RDI	Regulated Deficit Irrigation
RFID	Radio-Frequency Identification
SBA	Sidi Bel Abbès
SQL	Structured Query Language
SWB	Soil Water Balance
UAV	Unmanned Aerial Vehicle
UI	User Interface
WSN	Wireless Sensor Network

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
ملخص	v
List of Abbreviations	vi
General Introduction	1
I Theoretical Background	4
1 Precision Agriculture and Smart Irrigation	5
1.1 1.1 Introduction	5
1.2 1.2 Water scarcity and irrigation management	6
1.3 1.3 From conventional irrigation to smart irrigation	7
1.4 1.4 Main variables in smart irrigation	7
1.5 1.5 Communication technologies in smart irrigation	8
1.6 1.6 Actuators and irrigation control	9
1.7 1.7 Challenges of smart irrigation systems	10
1.8 1.8 Conclusion	10
2 Digital Twins, IoT, and Artificial Intelligence	12
2.1 2.1 Introduction	12
2.2 2.2 Digital Model, Digital Shadow, and Digital Twin	12
2.3 2.3 Digital Twin architecture layers	13
2.4 2.4 IoT as the physical-to-digital bridge	14
2.5 2.5 Artificial intelligence and machine learning in irrigation	14
2.6 2.6 Simulation and what-if analysis	15
2.7 2.7 Human-in-the-loop Digital Twins	16
2.8 2.8 Digital Twin challenges in agriculture	16
2.9 2.9 Conclusion	16
II State of the Art and Scientific Positioning	17
3 Digital Twins in Agriculture and Irrigation: State of the Art	18

3.1	3.1 Review methodology	18
3.2	3.2 General agricultural Digital Twin reviews	19
3.3	3.3 AI-enabled Digital Twins in agriculture	20
3.4	3.4 Digital Twins for agricultural water management	20
3.5	3.5 IoT-based smart irrigation reviews	21
3.6	3.6 Machine learning for smart irrigation	22
3.7	3.7 Directly related Digital Twin irrigation studies	22
3.8	3.8 Research gaps identified in the literature	23
3.9	3.9 Conclusion	23
4	Comparative Analysis and Scientific Positioning	24
4.1	4.1 Comparison criteria	24
4.2	4.2 Comparative matrix	25
4.3	4.3 Positioning of Agrio by Digital Twin maturity	26
4.4	4.4 Novelty of the proposed project	26
4.5	4.5 Weaknesses and limitations identified from the comparison	27
4.6	4.6 Conclusion	27
III	Requirements and Architecture	28
5	Requirements Specification and Design	29
5.1	5.1 Introduction	29
5.2	5.2 System objectives	29
5.3	5.3 Actors	29
5.4	5.4 Functional requirements	30
5.5	5.5 Non-functional requirements	30
5.6	5.6 Constraints	30
5.7	5.7 Use-case model	31
5.8	5.8 Activity workflow	33
5.9	5.9 Domain class diagrams	33
5.10	5.10 Conclusion	33
6	Global Architecture of the Proposed Digital Twin	34
6.1	6.1 Introduction	34
6.2	6.2 Architectural overview	34
6.3	6.3 Physical prototype layer	34
6.4	6.4 Sensing layer	35
6.5	6.5 Radio-frequency and gateway layer	35
6.6	6.6 MQTT communication layer	35
6.7	6.7 Backend layer	35
6.8	6.8 Database layer	36
6.9	6.9 Digital Twin layer	36
6.10	6.10 Machine learning layer	36
6.11	6.11 Recommendation and validation layer	36
6.12	6.12 Actuator and feedback layer	36
6.13	6.13 Deployment architecture	36
6.14	6.14 Discussion	37

6.15	6.15 Conclusion	37
IV	Implementation	38
7	Backend, Database, and IoT Data Ingestion	39
7.1	7.1 Introduction	39
7.2	7.2 FastAPI backend organization	39
7.3	7.3 API modules	39
7.4	7.4 MQTT payload structure	39
7.5	7.5 Parser and normalization	40
7.6	7.6 Device mapping and persistence	40
7.7	7.7 Real-time pipeline	40
7.8	7.8 PostgreSQL schema	40
7.9	7.9 Runtime sequence	41
7.10	7.10 Error handling and fallback	41
7.11	7.11 Conclusion	41
8	Digital Twin and Machine Learning Layer	43
8.1	8.1 Introduction	43
8.2	8.2 Virtual zone state	43
8.3	8.3 Dataset adaptation	43
8.4	8.4 Feature engineering	43
8.5	8.5 Random Forest models	44
8.6	8.6 ML inference service	44
8.7	8.7 Digital Twin state update	44
8.8	8.8 What-if simulation	44
8.9	8.9 Recommendation generation	45
8.10	8.10 Explainability	45
8.11	8.11 Limitations	45
8.12	8.12 Conclusion	45
9	User Interfaces and Irrigation Control	47
9.1	9.1 Introduction	47
9.2	9.2 Frontend architecture	47
9.3	9.3 Operations dashboard	47
9.4	9.4 Zone monitoring	47
9.5	9.5 AI recommendation preview	47
9.6	9.6 Pump and actuator control	48
9.7	9.7 Forecast workspace	48
9.8	9.8 Digital Twin simulation workspace	48
9.9	9.9 Schedules page	48
9.10	9.10 Validated decisions page	48
9.11	9.11 Mobile application	48
9.12	9.12 Conclusion	48

V	Deployment, Validation, and Discussion	51
10	Deployment Environment	52
10.1	10.1 Introduction	52
10.2	10.2 Docker Compose stack	52
10.3	10.3 PostgreSQL service	52
10.4	10.4 ML service	52
10.5	10.5 Backend service	52
10.6	10.6 Frontend service	53
10.7	10.7 Scheduler worker	53
10.8	10.8 Ngrok and mobile access	53
10.9	10.9 Deployment limitations	53
10.10	10.10 Conclusion	53
11	Results, Validation, and Discussion	55
11.1	11.1 Introduction	55
11.2	11.2 Prototype validation context	55
11.3	11.3 Sensor data acquisition	55
11.4	11.4 Database persistence	55
11.5	11.5 Dashboard synchronization	55
11.6	11.6 ML prediction validation	56
11.7	11.7 Simulation validation	56
11.8	11.8 Pump control validation	56
11.9	11.9 Discussion of results	56
11.10	11.10 Comparison with literature	56
11.11	11.11 Limitations	56
11.12	11.12 Future work	56
11.13	11.13 Conclusion	57
	General Conclusion	59
A	Figure Placement Checklist	62
B	Full Class Diagram	66
C	Full Detailed Architecture	67
D	Complete Database ERD	68
E	Example MQTT Payloads	69
F	Missing Visual Capture Guide	70
G	Thesis Expansion Notes for Final Editing	71
H	Detailed Paper-by-Paper Literature Matrix	72
H.1	Subeesh and Chauhan, 2026 [1]	72
H.2	Zhang et al., 2025 [2]	72
H.3	Gund et al., 2025 [3]	73
H.4	Awais et al., 2025 [4]	73

H.5	Tsaousidis et al., 2026 [5]	73
H.6	Ahsen et al., 2025 [6]	73
H.7	García et al., 2020 [7]	74
H.8	Obaideen et al., 2022 [8]	74
H.9	Del-Coco et al., 2024 [9]	74
H.10	Morchid et al., 2026 [10]	74
H.11	Kodali and Sarjerao, 2017 [11]	75
H.12	Balaceanu et al., 2019 [12]	75
H.13	Millán et al., 2025 [13]	75
H.14	Bellvert et al., 2023 [14]	75
H.15	Manocha et al., 2024 [15]	76
H.16	Tancredi et al., 2025 [16]	76
H.17	Qin et al., 2025 [17]	76
H.18	Liu et al., 2025 [18]	76
H.19	El Hachimi et al., 2025 [19]	77
I	Detailed Database Table Descriptions	78
I.1	I.1 Why this database supports a Digital Twin	80
J	Detailed API and Service Responsibilities	81
J.1	J.1 Internal services	82
J.2	J.2 API evidence to collect	82
K	Detailed Algorithms and Pseudocode	83
K.1	Algorithm K.1 - MQTT sensor ingestion	83
K.2	Algorithm K.2 - Real-time Digital Twin update	83
K.3	Algorithm K.3 - What-if simulation	84
K.4	Algorithm K.4 - Recommendation validation	84
K.5	Algorithm K.5 - Scheduled irrigation execution	84
K.6	K.6 Formula notes for simple moisture projection	85
L	Expanded Chapter Text Blocks for Final Integration	86
L.1	L - Precision agriculture as a systems problem	86
L.2	L - Why MQTT is appropriate for Agrio	86
L.3	L - Why a relational database is useful	86
L.4	L - Why Random Forest is a pragmatic first model	87
L.5	L - Why human validation matters	87
L.6	L - Why the prototype is still valuable without field-scale validation	87
M	Defense-Oriented Scientific Positioning	88
M.1	M.1 If the jury asks: Why is this a Digital Twin?	88
M.2	M.2 If the jury asks: What is the novelty?	88
M.3	M.3 If the jury asks: What are the weaknesses?	88
M.4	M.4 If the jury asks: Why combine Master 2 and Engineering?	89

List of Figures

1	Global view of the thesis methodology from literature review to prototype validation. Source: Authors.	3
1.1	Conceptual scope of precision agriculture and smart irrigation. Source: Authors. . .	6
1.2	Generic communication chain for IoT-based smart irrigation. Source: Authors. . . .	9
1.3	Pump control concept using WeMos, relay, and MQTT command/status topics. Source: Authors.	10
2.1	Digital Model, Digital Shadow, and Digital Twin maturity levels. Source: Authors. .	13
2.2	Layered architecture of an agricultural Digital Twin. Source: Authors.	14
3.1	Literature review methodology and thematic classification. Source: Authors.	19
3.2	Example of agricultural Digital Twin overview from literature [insert or redraw based on Subeesh and Chauhan, Fig. 1]. Source: Adapted/redrawn from [1].	20
3.3	Generic smart irrigation system architecture from ML literature [insert/redraw based on Del-Coco et al., Fig. 1]. Source: Adapted/redrawn from [9].	21
4.1	Positioning of Agrio between Digital Model, Digital Shadow, and operational Digital Twin. Source: Authors.	26
5.1	Use-case diagram of the proposed Digital Twin smart irrigation system. Source: Authors.	32
5.2	Activity diagram of sensor ingestion, Digital Twin update, AI decision-making, and pump control. Source: Authors.	33
5.3	Agricultural domain class diagram representing farms, plots, zones, soil profiles, crops, and irrigation networks. Source: Authors.	33
5.4	Sensor, weather, and forecasting data model used by the smart irrigation platform. Source: Authors.	33
5.5	Digital Twin, prediction, and simulation data model. Source: Authors.	33
5.6	Decision, recommendation, scheduling, command, and feedback data model. Source: Authors.	33
6.1	Global overview of the Agrio Digital Twin smart irrigation architecture. Source: Authors.	34
6.2	Physical and communication architecture of the controlled smart irrigation prototype. Source: Authors.	35
6.3	MQTT topic architecture for sensor ingestion, actuator commands, and WeMos status feedback. Source: Authors.	35
6.4	Internal backend architecture of the FastAPI-based Digital Twin platform. Source: Authors.	35

6.5	Digital Twin processing architecture from live sensor data to simulation and recommendation. Source: Authors.	36
6.6	Docker Compose deployment architecture of the Agrio platform. Source: Authors. .	36
6.7	Detailed technical architecture of the implemented Digital Twin smart irrigation platform. Source: Authors.	37
7.1	Example MQTT payloads used for sensor ingestion, WeMos status feedback, and actuator commands. Source: Authors.	40
7.2	ERD of the agricultural structure tables. Source: Authors.	40
7.3	ERD of the sensor, weather, and forecasting tables. Source: Authors.	40
7.4	ERD of the Digital Twin, prediction, and simulation tables. Source: Authors. . . .	40
7.5	ERD of the recommendation, scheduling, command, and feedback tables. Source: Authors.	41
7.6	Runtime sequence of MQTT data ingestion, ML prediction, dashboard polling, and WeMos actuator command. Source: Authors.	41
7.7	Live MQTT payloads received through HiveMQ for sensor data and WeMos status. Source: Authors.	41
7.8	FastAPI OpenAPI documentation exposing the REST endpoints of the backend. Source: Authors.	41
7.9	PostgreSQL stored sensor readings persisted from MQTT-originated measurements. Source: Authors.	41
8.1	Dataset adaptation and feature engineering workflow used before model training. Source: Authors.	43
8.2	Machine learning pipeline from dataset adaptation to Random Forest inference. Source: Authors.	44
8.3	ML service health endpoint showing model loading status. Source: Authors.	44
8.4	ML prediction response for irrigation need and suggested irrigation hour. Source: Authors.	44
8.5	Digital Twin state update flow from sensor readings to synchronized zone state. Source: Authors.	44
8.6	What-if simulation logic comparing no-irrigation, with-irrigation, and ML-selected paths. Source: Authors.	45
8.7	Feature importance of the Random Forest need_irrigation model. Source: Authors.	46
8.8	Feature importance of the Random Forest suggested_hour model. Source: Authors.	46
8.9	Confusion matrix of the need_irrigation Random Forest model. Source: Authors. .	46
8.10	Confusion matrix of the suggested_hour Random Forest model. Source: Authors. .	46
9.1	Agrio operations dashboard showing MQTT connection, active zone coverage, moisture snapshot, sensor health, and Digital Twin status. Source: Authors.	48
9.2	Zone selection panel for inspecting the Digital Twin state, recommendations, commands, and historical data. Source: Authors.	49
9.3	Current field status and AI recommendation preview for the monitored irrigation zone. Source: Authors.	49
9.4	Pump and actuator control interface with WeMos status and weather station summary. Source: Authors.	49
9.5	Forecast workspace used to generate weather-aware irrigation planning decisions. Source: Authors.	49

9.6	Digital Twin simulation workspace for running what-if irrigation scenarios. Source: Authors.	49
9.7	Moisture projection chart comparing no-irrigation, with-irrigation, and ML-selected trajectories. Source: Authors.	49
9.8	Irrigation schedules page for planning and managing pump execution events. Source: Authors.	49
9.9	Validated decisions interface for tracking approved irrigation recommendations. Source: Authors.	49
9.10	Mobile application screens providing access to monitoring and irrigation decision functions. Source: Authors.	50
10.1	Docker Compose service organization of the deployed Agrio platform. Source: Authors.	53
10.2	Running Docker containers of the deployed Digital Twin smart irrigation platform. Source: Authors.	53
10.3	Deployment communication flow between frontend, backend, database, ML service, MQTT broker, and external APIs. Source: Authors.	53
10.4	Ngrok tunnel used to expose local services for mobile access during prototype testing. Source: Authors.	53
11.1	Sensor MQTT message received on the sensors/data topic. Source: Authors.	57
11.2	Database persistence proof showing stored sensor readings from MQTT-originated data. Source: Authors.	57
11.3	Dashboard update reflecting the latest stored sensor and Digital Twin data. Source: Authors.	57
11.4	ML prediction result showing irrigation need, suggested hour, confidence, and action. Source: Authors.	58
11.5	Digital Twin simulation result comparing projected moisture and recommended action. Source: Authors.	58
11.6	Pump ON command sent to the WeMos actuator through MQTT. Source: Authors.	58
11.7	WeMos status feedback after pump activation. Source: Authors.	58
11.8	Pump OFF command sent to the WeMos actuator through MQTT. Source: Authors.	58
11.9	Execution feedback or irrigation event status recorded after pump command execution. Source: Authors.	58
B.1	Complete class diagram of the implemented Digital Twin smart irrigation platform. Source: Authors.	66
C.1	Large detailed architecture of the implemented Agrio platform. Source: Authors.	67
D.1	Complete database ERD of the smart_irrigation schema. Source: Authors.	68

List of Tables

1	Abbreviations used in the thesis	vi
2	Summary of thesis objectives and contributions	3
1.1	Evolution from conventional irrigation to Digital Twin irrigation	7
1.2	Measured variables and their role in Agrio	8
2.1	Machine learning algorithms used in smart irrigation literature	15
3.1	Key literature themes used in this thesis	23
4.1	Comparison criteria for Digital Twin irrigation systems	24
4.2	Scientific comparison of selected works with Agrio	25
4.3	Novelty and contribution of Agrio	26
5.1	Functional requirements	30
5.2	Non-functional requirements	31
6.1	Architecture layers and implementation choices	37
7.1	Main database groups	42
8.1	Features used by the need_irrigation model	45
8.2	Features used by the suggested_hour model	46
8.3	Model performance metrics from project metadata	46
10.1	Deployment services and roles	54
11.1	Test scenario summary	57
I.1	Detailed description of database tables	78
J.1	Backend API responsibilities	81
J.2	Internal backend service responsibilities	82

General Introduction

Agriculture is one of the foundations of food security, economic development, and social stability. However, it is also one of the sectors most exposed to climate variability, water scarcity, soil degradation, and increasing production pressure. In many regions, irrigation remains a central factor determining crop quality and yield. When irrigation is applied too late, plants may suffer water stress and productivity may decrease. When irrigation is applied too early or in excessive quantities, water is wasted, soil nutrients may leach, and the cost of operation increases. The challenge is therefore not simply to irrigate, but to irrigate at the right time, with the right quantity, and according to the actual state of the soil, crop, and environment.

Traditional irrigation practices are often based on fixed schedules, manual field observation, or the farmer's experience. These practices remain valuable because farmers possess contextual knowledge of their crops and land. Nevertheless, they can become insufficient when environmental conditions change quickly or when water availability becomes uncertain. Precision agriculture responds to this issue by relying on sensors, communication systems, data processing, and intelligent decision support. Smart irrigation is one of the most important applications of precision agriculture because it directly addresses water-use efficiency and sustainability.

The emergence of the Internet of Things (IoT), wireless sensor networks, low-cost microcontrollers, cloud services, and artificial intelligence has transformed agricultural monitoring. Sensors can measure soil moisture, soil temperature, air temperature, relative humidity, rainfall, wind speed, wind direction, pressure, leaf wetness, and light intensity. Communication systems can transmit these measurements to a gateway or cloud platform. Machine learning models can process data and predict whether irrigation is required. Dashboards can visualize the current state of a plot and allow the farmer to control an actuator remotely. However, many smart irrigation projects remain limited to monitoring, threshold-based control, or isolated prediction models.

Digital Twin technology offers a more integrated approach. A Digital Twin is not only a dashboard or a static simulation. It is a virtual representation of a physical system, continuously updated using real data, able to simulate future behavior, support decision-making, and potentially influence the physical system through feedback or actuation. In agriculture, Digital Twins can represent farms, plots, irrigation zones, greenhouses, plants, soil profiles, livestock, or agricultural machinery. Their value lies in connecting physical measurements, computational models, prediction algorithms, and operational decisions within a coherent cyber-physical loop.

This thesis presents the design and deployment of Agrio, a Digital Twin-based intelligent irrigation platform for precision agriculture. The system was developed as a combined Master 2 and Engineering degree project. The Master 2 dimension focuses on the research context, the state of the art, the comparison of Digital Twins in agriculture, and the scientific positioning of the proposed work. The engineering dimension focuses on the design, implementation, deployment, and validation of a complete prototype integrating physical sensors, radio-frequency communication, IoT

messaging, backend processing, database storage, machine learning inference, Digital Twin simulation, web/mobile interfaces, scheduling, and pump control.

The physical prototype is based on a controlled experimental smart farming setup. It uses a Libelium Smart Agriculture kit, XBee radio-frequency communication, an XBee gateway connected to a Raspberry Pi, HiveMQ/MQTT for cloud messaging, a WeMos actuator, and a water pump. Sensor data are received by the Raspberry Pi gateway, decoded or decrypted when required, converted into JSON payloads, and published to MQTT. The backend subscribes to the sensor topic, parses the payload, normalizes the values, stores sensor readings in PostgreSQL, updates the Digital Twin state, calls the machine learning service, generates recommendations, and exposes the results through REST APIs consumed by the dashboard and mobile application.

The project is named Agrio. Its web dashboard provides an operational view of the monitored plot, including active zone coverage, moisture snapshot, sensor health, Digital Twin status, AI recommendation preview, weather summary, pump status, manual control, forecasting, schedules, validated decisions, and simulation workspace. The Digital Twin simulation page allows the user to compare the expected evolution of soil moisture with and without irrigation over a selected horizon. This makes the system more advanced than a simple IoT dashboard, because it combines live monitoring, predictive inference, what-if simulation, human validation, and actuation.

The central research question of this thesis is: how can a Digital Twin architecture be designed and deployed to improve decision support and operational control in an intelligent irrigation system? This question is divided into secondary questions: what are the main concepts and maturity levels of Digital Twins in agriculture? Which technologies are commonly used in IoT-based smart irrigation systems? How can machine learning support irrigation decision-making? What are the strengths and limitations of current research? Where does the proposed Agrio prototype stand compared with existing works? What implementation choices are necessary to connect the physical and virtual irrigation system?

The objectives of the project are: to study Digital Twins and smart irrigation systems in the agricultural domain; to design a complete architecture for an intelligent irrigation Digital Twin; to implement IoT data ingestion using MQTT; to store agricultural measurements and system states in a structured database; to integrate machine learning for irrigation prediction; to simulate moisture evolution under alternative scenarios; to generate irrigation recommendations; to support human validation and scheduling; and to control a physical water pump through a WeMos device.

The main contributions are both scientific and technical. Scientifically, the thesis provides a structured review and comparison of Digital Twins, IoT-based smart irrigation, and machine learning irrigation systems. Technically, the project implements an end-to-end prototype connecting Libelium sensors, XBee RF, Raspberry Pi, MQTT, FastAPI, PostgreSQL, Random Forest inference, Digital Twin simulation, web/mobile interfaces, and pump control. The project therefore demonstrates a practical path from research concepts to an operational cyber-physical prototype.

The thesis is organized into eleven chapters. Chapter 1 introduces precision agriculture and smart irrigation. Chapter 2 presents Digital Twins, IoT, and artificial intelligence. Chapter 3 reviews the state of the art of Digital Twins in agriculture and irrigation. Chapter 4 compares existing works and positions the proposed contribution. Chapter 5 presents requirements and design. Chapter 6 details the global architecture. Chapter 7 explains backend, database, and IoT ingestion. Chapter 8 presents the Digital Twin and machine learning layer. Chapter 9 describes user interfaces and irrigation control. Chapter 10 explains deployment. Chapter 11 presents validation, discussion, limitations, and perspectives.

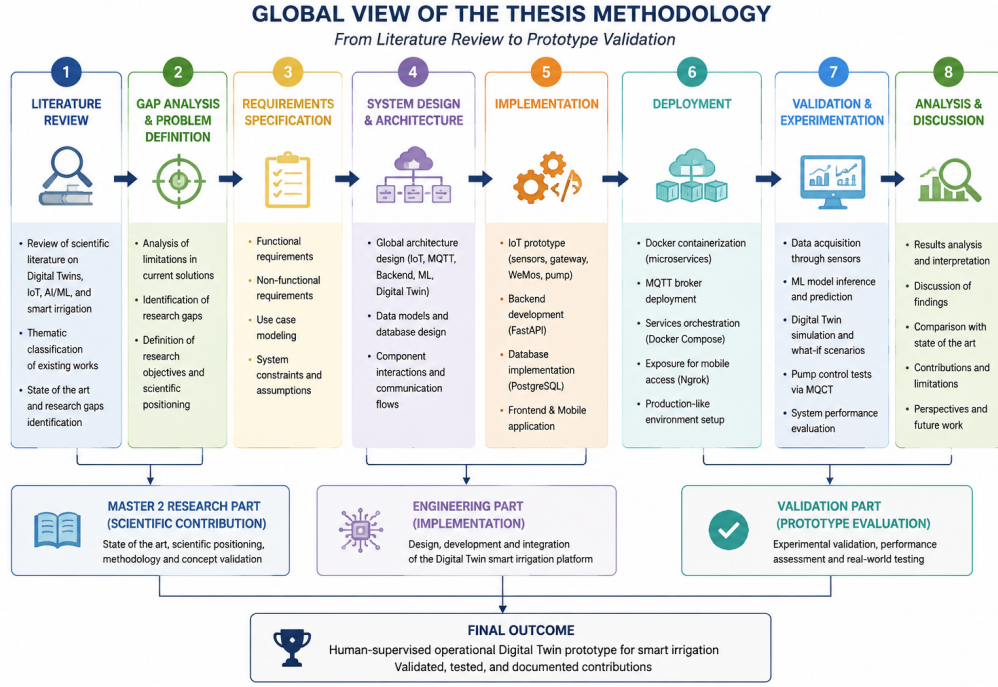


Figure 1: Global view of the thesis methodology from literature review to prototype validation.
Source: Authors.

Table 2: Summary of thesis objectives and contributions

Dimension	Objective	Contribution
Scientific	Study Digital Twins, smart irrigation, IoT, and ML literature	State-of-the-art review, comparison matrix, research gap identification
Architectural	Design an end-to-end Digital Twin irrigation architecture	Layered system connecting sensing, MQTT, backend, database, ML, simulation, dashboard, and pump
Implementation	Build the Agrio prototype	FastAPI, PostgreSQL, HiveMQ/MQTT, Random Forest service, Next.js dashboard, Flutter app, WeMos control
Validation	Demonstrate the technical workflow	Evidence through MQTT payloads, database readings, dashboard updates, simulation outputs, and actuator status

Part I

Theoretical Background

Chapter 1

Precision Agriculture and Smart Irrigation

1.1 1.1 Introduction

Precision agriculture is a management approach that uses data and technology to adapt agricultural practices to spatial and temporal variability. Instead of treating a field as a uniform entity, precision agriculture recognizes that soil properties, water availability, crop development, microclimate, pest pressure, and nutrient status can differ across zones and over time. This approach is especially important for irrigation, where uniform schedules may over-irrigate some areas and under-irrigate others.

Smart irrigation is the application of sensing, communication, computation, and control to irrigation management. It aims to apply water according to actual crop and soil needs. In the literature, smart irrigation systems are often described as combinations of sensors, wireless networks, cloud platforms, control units, data processing, and user interfaces. García et al. [7] review sensors, nodes, wireless technologies, and irrigation actuators used in IoT-based irrigation systems, while Obaideen et al. [8] emphasize the role of soil and weather monitoring in achieving water-use efficiency.

In this thesis, smart irrigation is understood as an operational system that can measure relevant variables, transmit them to a processing layer, infer the irrigation need, and support or execute irrigation decisions. The proposed Agrio platform follows this definition but extends it through a Digital Twin layer that keeps a virtual state of the irrigation zone and simulates future moisture evolution.

CONCEPTUAL SCOPE OF PRECISION AGRICULTURE AND SMART IRRIGATION

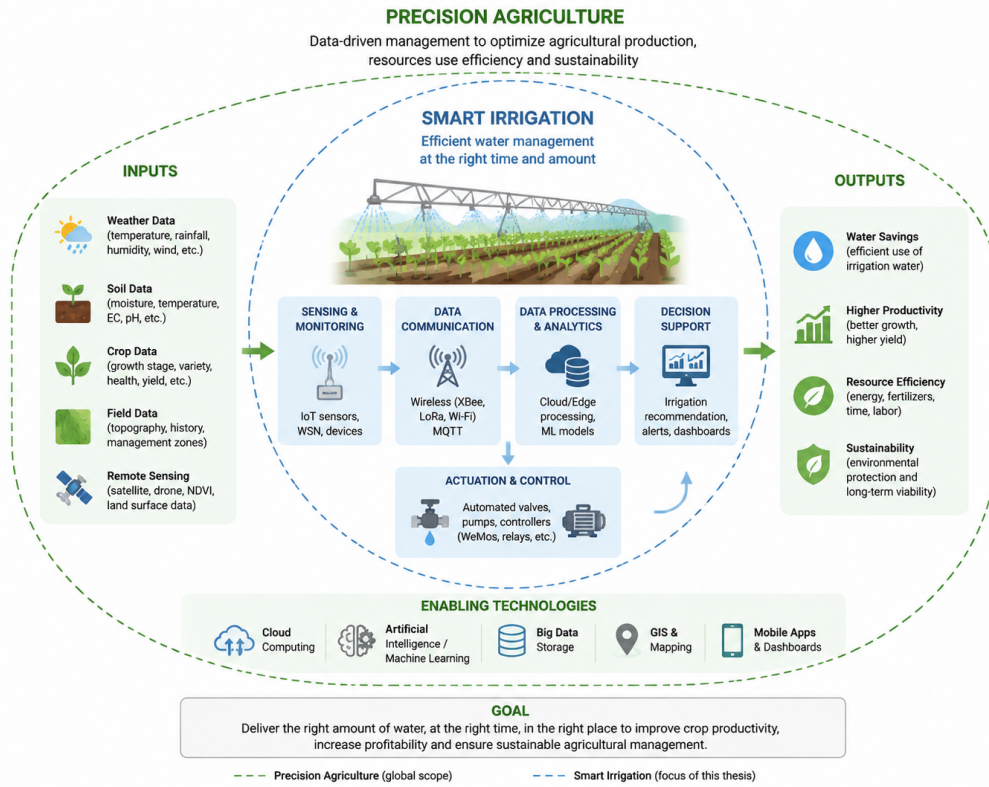


Figure 1.1: Conceptual scope of precision agriculture and smart irrigation. Source: Authors.

1.2 1.2 Water scarcity and irrigation management

Water scarcity is one of the main motivations behind intelligent irrigation research. Several reviewed papers remind that agriculture accounts for a large share of global freshwater withdrawals, often around 70%. This fact is used in many irrigation and Digital Twin studies to justify the need for more efficient water management. The issue is not only environmental; it is also economic and social. Water scarcity can reduce yields, increase food prices, and make agricultural production more vulnerable to drought and climate variability.

Irrigation scheduling is the process of deciding when and how much water should be applied. Classical scheduling may rely on fixed calendars, farmer experience, empirical observation of soil moisture, crop coefficients, or soil water balance methods. These methods can be effective when they are correctly calibrated, but they may become inaccurate when weather, soil, crop stage, or irrigation infrastructure changes. In real agricultural environments, local conditions can cause deviations between estimated and actual crop water consumption.

In the context of this project, the water management problem is addressed at prototype scale. The objective is not to claim immediate large-scale water saving, but to design and validate a technical architecture capable of supporting precise irrigation decisions. This distinction is important academically: the prototype proves the feasibility of the Digital Twin workflow, while future field trials are required to quantify agronomic and economic water savings under real farm conditions.

1.3 1.3 From conventional irrigation to smart irrigation

Conventional irrigation methods include flood irrigation, furrow irrigation, sprinkler irrigation, drip irrigation, and manual watering. Each method has advantages and constraints depending on crop type, soil texture, topography, water availability, and farm infrastructure. Drip irrigation is often considered efficient because it delivers water close to the root zone, but even drip systems can waste water if scheduling is poorly adjusted.

IoT-based smart irrigation introduces sensing and automation into this process. Soil moisture sensors can indicate whether the root zone contains sufficient water. Temperature and humidity sensors help estimate environmental demand. Rainfall sensors and weather forecasts help avoid unnecessary irrigation before or after rainfall. Wind and radiation information can support evapotranspiration estimation. Actuators such as pumps, valves, and relays execute irrigation actions.

García et al. [7] show that many IoT irrigation systems use pumps or valves, soil moisture sensors, microcontrollers, and wireless communication. Obaideen et al. [8] explain that smart irrigation includes data acquisition, irrigation control, wireless communication, data processing, and fault detection. These components are also present in Agrio, but the project adds Digital Twin state management, machine learning prediction, scenario simulation, and user validation.

Table 1.1: Evolution from conventional irrigation to Digital Twin irrigation

Approach	Decision basis	Strength	Limitation
Manual irrigation	Farmer observation and experience	Simple and context-aware	Difficult to scale and may miss rapid changes
Scheduled irrigation	Calendar and fixed duration	Easy to implement	Insensitive to actual soil moisture and weather
Sensor-triggered irrigation	Thresholds on soil moisture or weather	Responsive to measured data	Usually limited to current state and simple rules
ML-assisted irrigation	Historical and real-time features	Learns complex patterns	Requires data quality, model validation, and interpretability
Digital Twin irrigation	Synchronized virtual state, prediction, simulation, feedback	Supports what-if analysis and decision traceability	Requires integration, calibration, and cyber-physical reliability

1.4 1.4 Main variables in smart irrigation

The effectiveness of a smart irrigation system depends on the variables it monitors. Soil moisture is the central variable because it indicates water availability in the root zone. Soil temperature influences root activity and water uptake. Air temperature, humidity, wind speed, and light intensity influence evapotranspiration. Rainfall indicators determine whether natural precipitation can replace irrigation. Leaf wetness can support crop health monitoring and disease risk assessment.

The Libelium Smart Agriculture kit used in this project supports a wide range of environmental and agricultural measurements. The MQTT payloads generated by the system include soil moisture, soil temperature, air temperature, humidity, pressure, leaf wetness, light intensity, rainfall current/hour/day, wind direction, wind speed, timestamp, and device identifier. This multi-variable sensing approach is important because irrigation decisions should not be based on soil moisture alone.

However, more variables also imply more processing complexity. Sensor values may arrive in different formats, may contain missing values, or may require normalization. For example, soil

moisture may be sent as a fraction and converted to percentage; light intensity may include suffixes that must be parsed; wind direction may be categorical and converted into degrees or cyclic features. These issues are handled by the backend parser and dataset adaptation pipeline.

Table 1.2: Measured variables and their role in Agrio

Variable	Role in irrigation decision	Used in Agrio
Soil moisture	Primary indicator of water availability	Yes
Soil temperature	Influences root activity and soil water dynamics	Yes
Air temperature	Supports evapotranspiration and stress estimation	Yes
Humidity	Indicates atmospheric demand and evaporation conditions	Yes
Rainfall	Avoids unnecessary irrigation and supports forecast decisions	Yes
Wind speed/direction	Influences evapotranspiration and environmental interpretation	Yes
Light intensity	Supports environmental context and crop-energy conditions	Yes
Leaf wetness	Supports plant/environment monitoring	Yes
Pressure	Additional weather/environment indicator	Yes
Pump status	Closes the loop between decision and actuation	Yes

1.5 1.5 Communication technologies in smart irrigation

Smart irrigation requires reliable communication between sensors, gateways, cloud services, and actuators. The literature reports many possible communication technologies: Wi-Fi, Bluetooth, Zigbee, XBee, LoRa, LoRaWAN, GSM/GPRS, NB-IoT, and cellular networks. The best choice depends on field size, power constraints, range, data rate, cost, and infrastructure availability.

In Agrio, the field-side communication uses XBee radio-frequency communication. The XBee gateway receives the transmitted sensor data, and a Raspberry Pi processes the received frames. The gateway then publishes a standardized JSON payload to HiveMQ using MQTT. MQTT is well adapted to IoT communication because it follows a publish/subscribe model and supports decoupling between data producers and consumers. The backend subscribes to sensor topics, while the WeMos actuator subscribes to command topics and publishes status feedback.

This layered communication approach is important because it separates the field hardware protocol from the cloud/backend protocol. The backend does not need to interpret raw radio frames; it only consumes normalized MQTT payloads. This makes the architecture more modular and easier to extend to other sensor stations or communication technologies in the future.

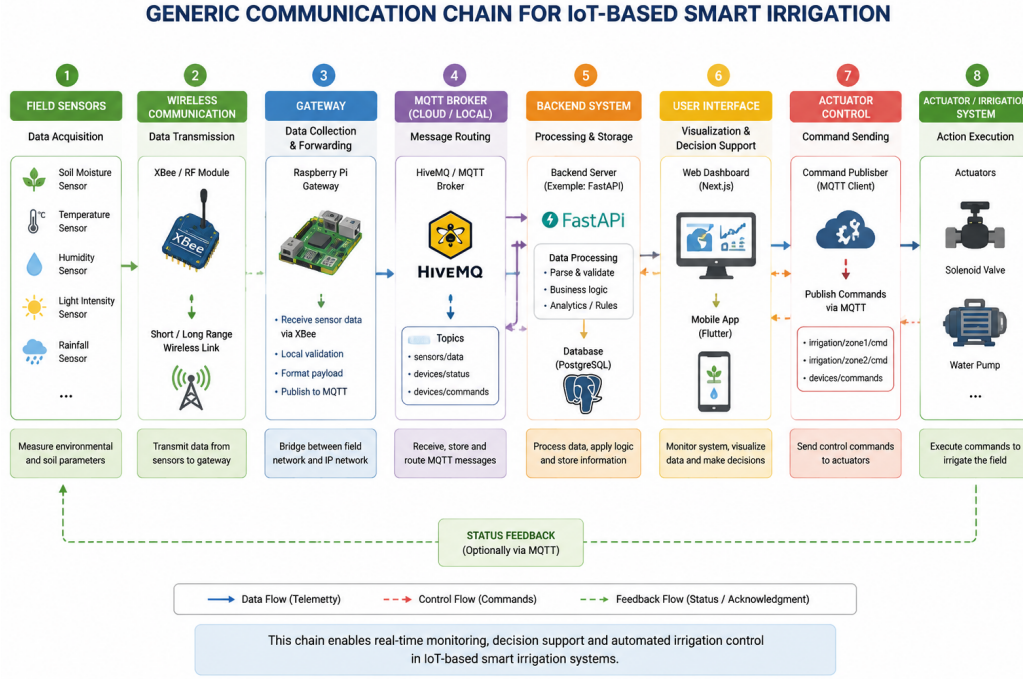


Figure 1.2: Generic communication chain for IoT-based smart irrigation. Source: Authors.

1.6 1.6 Actuators and irrigation control

Actuators transform digital decisions into physical irrigation actions. In irrigation systems, common actuators include pumps, solenoid valves, relays, fertigation injectors, and motorized valves. In a smart irrigation architecture, actuation must be carefully controlled because an incorrect command can waste water or damage the prototype. Therefore, the system must distinguish between recommendation, validation, scheduling, and physical execution.

The Agrio prototype uses a WeMos device to control the pump through digital pins. In the current implementation, D5 is used for pump control and D3 is used for an auxiliary lamp. The backend publishes MQTT commands such as ON_D5 and OFF_D5 to the WeMos command topic. The WeMos publishes status messages indicating the state of D5 and D3. The dashboard reads the status through the backend and allows manual control or validated scheduled operation.

The system intentionally separates simulation from actuation. Running a Digital Twin simulation does not start the pump. Simulation only predicts a virtual outcome. Physical actuation occurs through explicit manual command or scheduled execution after validation. This design choice improves safety and makes the prototype more acceptable for academic demonstration.

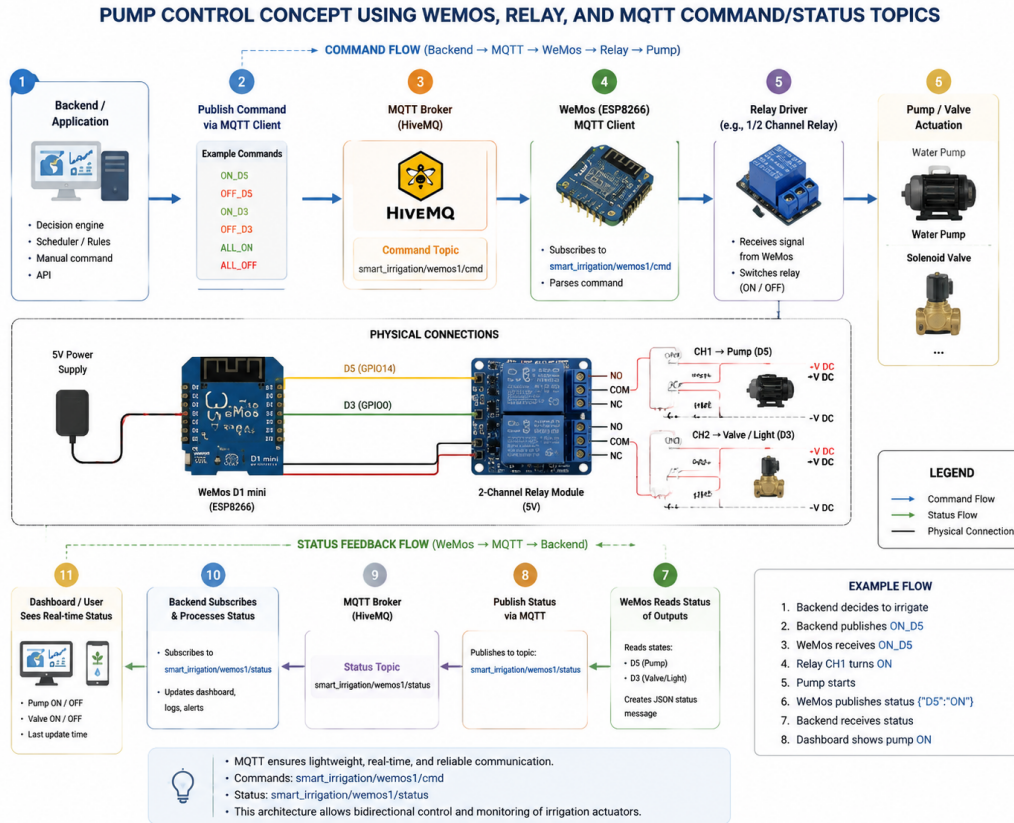


Figure 1.3: Pump control concept using WeMos, relay, and MQTT command/status topics. Source: Authors.

1.7 1.7 Challenges of smart irrigation systems

The literature identifies several challenges that limit the adoption of smart irrigation: high sensor cost, sensor calibration, communication reliability, power management, data security, interoperability, model generalization, farmer acceptance, and limited long-term validation. The agricultural environment is also more variable than many industrial environments. Soil heterogeneity, crop growth, weather uncertainty, and biological responses make prediction and control difficult.

Many existing systems demonstrate promising results in laboratory or small-scale settings but lack extended field testing. This limitation is also relevant for Agrio. The current prototype demonstrates the complete technical workflow in a controlled environment, but long-term agronomic validation in real farm conditions remains future work. A rigorous academic thesis should therefore present both the strengths and limitations of the project transparently.

Another challenge is the gap between decision support and autonomous control. Fully automatic irrigation may be attractive, but it requires strong safety, calibration, and trust. Human-in-the-loop validation is a practical compromise for a final-year prototype: the system can recommend and simulate, while the user remains responsible for confirming or scheduling physical irrigation.

1.8 1.8 Conclusion

This chapter presented the context of precision agriculture and smart irrigation. It showed that irrigation optimization requires sensing, communication, data processing, decision support, and actuator

control. The chapter also introduced the importance of soil moisture, weather variables, wireless communication, and safe pump control. These concepts prepare the transition toward Digital Twin technology, which is presented in the next chapter as the conceptual foundation of Agrio.

Chapter 2

Digital Twins, IoT, and Artificial Intelligence

2.1 2.1 Introduction

Digital Twin technology has become a key concept in Industry 4.0 and is increasingly studied in agriculture. The term refers to a virtual counterpart of a physical asset, process, or system. Unlike a static digital model, a Digital Twin is connected to the physical system through data flows and can be updated according to the real state of the asset. In advanced cases, the Digital Twin can also influence the physical system through recommendations or actuation.

Agriculture is a challenging domain for Digital Twins because it includes living organisms, heterogeneous soils, uncertain weather, spatial variability, and economic constraints. Nevertheless, recent reviews agree that Digital Twins can improve monitoring, prediction, decision-making, resource optimization, and sustainability in smart farming [1]-[6]. This thesis focuses specifically on Digital Twins for intelligent irrigation.

2.2 2.2 Digital Model, Digital Shadow, and Digital Twin

A useful distinction in the literature is the separation between Digital Model, Digital Shadow, and Digital Twin. A Digital Model is a static or offline representation of a physical system. It may simulate crop growth or water balance using historical or manually entered data, but it is not automatically updated by the physical system. A Digital Shadow introduces a one-way data flow from the physical system to the virtual system. Sensor data can update the digital representation, but the virtual system does not directly influence the physical system. A Digital Twin adds bidirectional interaction: the physical system updates the virtual model, and the virtual model can support or trigger decisions that affect the physical system.

This distinction is important for positioning Agrio. The system is more than a Digital Model because it receives real MQTT data from the physical prototype. It is also more advanced than a simple Digital Shadow because it supports recommendation, validation, scheduling, and pump control. However, because full autonomous control is intentionally restricted for safety, Agrio can be described as a human-supervised operational Digital Twin prototype rather than a fully autonomous industrial Digital Twin.

Figure 2.1 should show the three maturity levels. It can be adapted conceptually from the

literature, but it is preferable to redraw it in an original style for the thesis to avoid copyright issues.

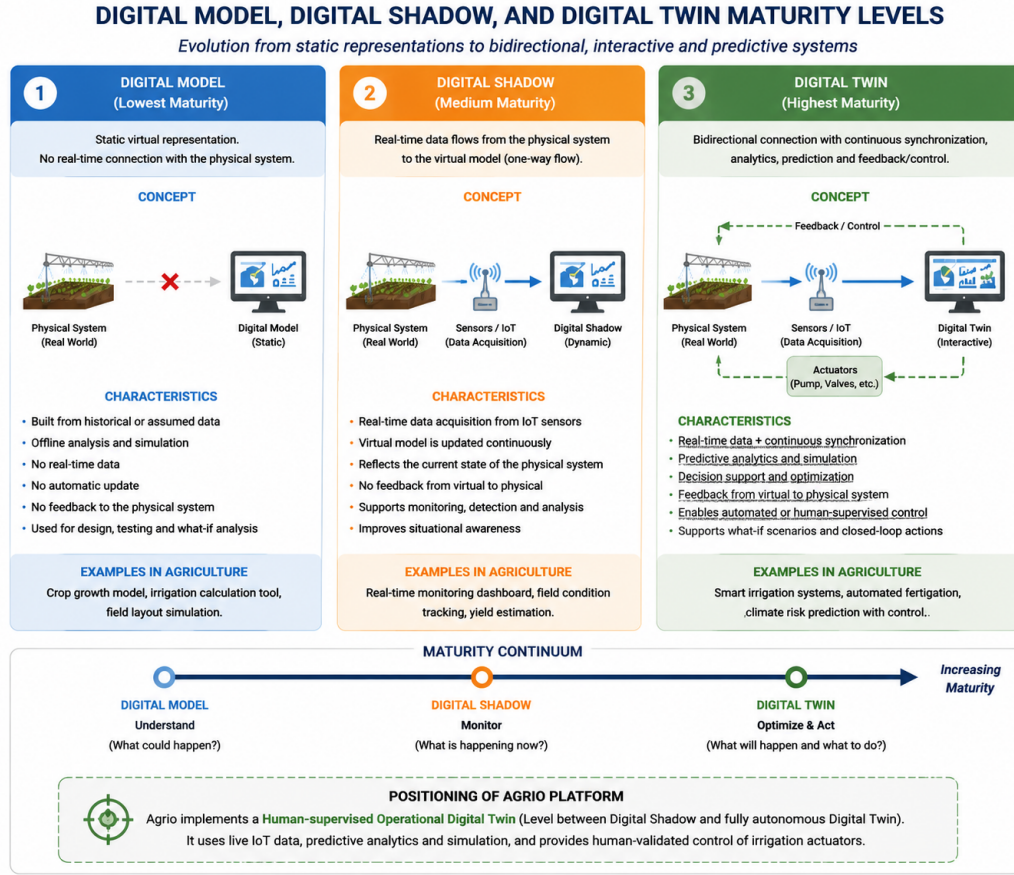


Figure 2.1: Digital Model, Digital Shadow, and Digital Twin maturity levels. Source: Authors.

2.3 Digital Twin architecture layers

Most Digital Twin architectures include a physical layer, sensing layer, communication layer, data management layer, modeling and simulation layer, analytics or intelligence layer, application layer, and feedback/control layer. In agriculture, the physical layer may include crops, soil, irrigation equipment, greenhouse systems, livestock, or machinery. The sensing layer collects data from sensors, cameras, satellites, drones, or weather stations. The communication layer transfers the data using wireless or wired protocols. The data layer stores and organizes the information. The modeling layer builds the virtual representation. The intelligence layer predicts, optimizes, or recommends. The application layer exposes the information to users. The feedback layer sends instructions to actuators or human operators.

Agrio implements these layers through concrete technologies. The physical layer is the controlled plant prototype with irrigation pipes and pump. The sensing layer is the Libelium Smart Agriculture kit. The communication layer includes XBee, Raspberry Pi gateway processing, and HiveMQ/MQTT. The data layer uses PostgreSQL. The backend and Digital Twin services create zone states and histories. The ML service predicts irrigation need and suggested hour. The dashboard and mobile application present information to the user. The WeMos and pump represent the actuator layer.

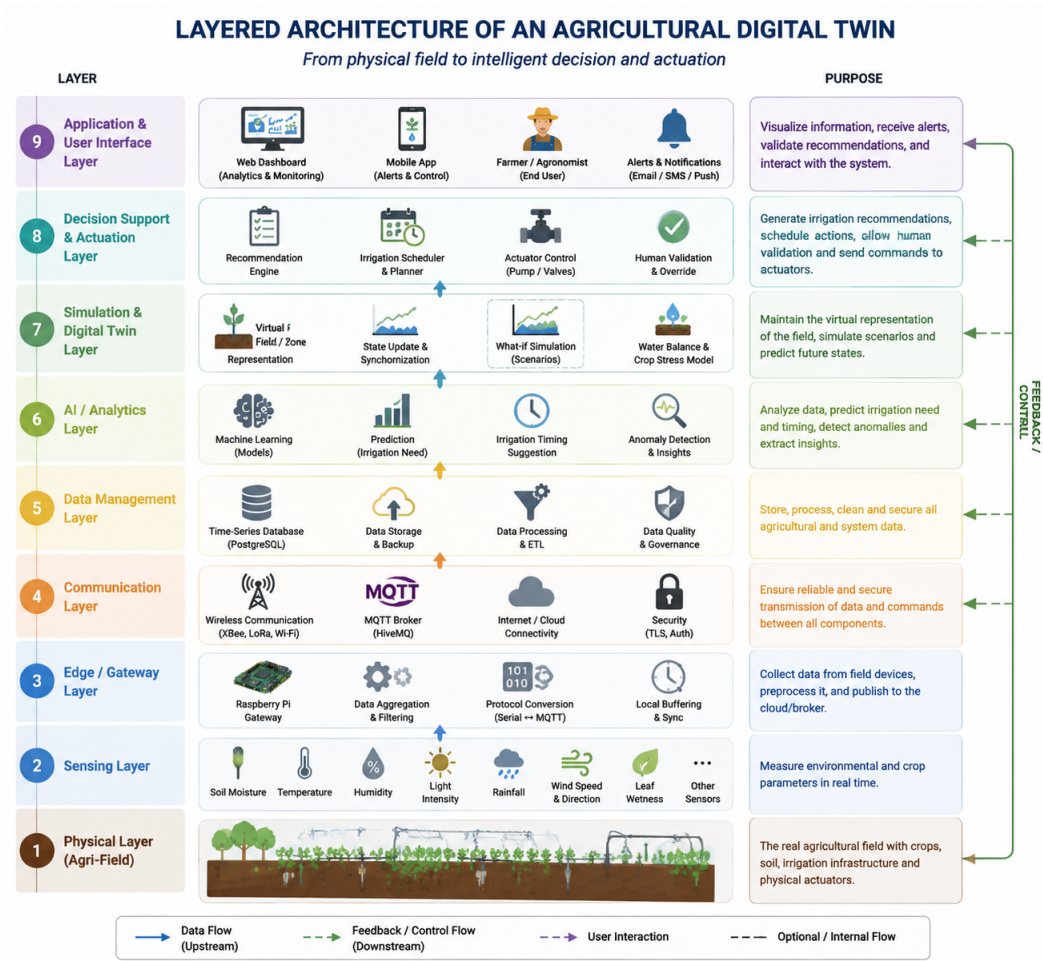


Figure 2.2: Layered architecture of an agricultural Digital Twin. Source: Authors.

2.4 2.4 IoT as the physical-to-digital bridge

IoT technologies are essential to Digital Twins because they provide the connection between physical assets and virtual models. Without live or near-real-time data, a Digital Twin remains a simulation or a static model. In smart irrigation, IoT devices collect soil and environmental data, transmit them to cloud or edge services, and support remote control of pumps or valves.

MQTT is commonly used in IoT systems because it follows a publish/subscribe communication pattern. Devices publish messages to topics, and subscribers receive messages from those topics. This decoupling is useful in Agrio because the gateway, backend, dashboard, and WeMos actuator do not need to communicate through direct synchronous calls. Instead, they exchange information through topics such as `sensors/data`, `smart_irrigation/wemos1/status`, and `smart_irrigation/wemos1/cmd`.

2.5 2.5 Artificial intelligence and machine learning in irrigation

Machine learning supports irrigation decision-making by learning patterns from historical and real-time data. Models can estimate soil moisture, predict evapotranspiration, classify irrigation need,

suggest irrigation timing, or optimize water amount. Del-Coco et al. [9] describe smart irrigation as a combination of sensors, IoT technologies, algorithms, datasets, and decision layers. They also emphasize that ML-based smart irrigation requires reliable datasets and careful evaluation.

Common algorithms include Decision Trees, Random Forests, Support Vector Machines, Gradient Boosting, Artificial Neural Networks, LSTM networks, and hybrid models. Random Forest is attractive for engineering prototypes because it is robust, handles nonlinear relationships, can work with mixed feature sets, and is less difficult to train than deep learning models. In Agrio, Random Forest models are used for two tasks: predicting whether irrigation is needed and suggesting an irrigation hour.

The thesis must be careful when presenting ML results. The metadata of the trained models can report high accuracy, but these results are training/evaluation metrics on the available dataset, not guaranteed field performance. Real agronomic validation would require long-term field experiments under different crops, soils, and weather conditions.

Table 2.1: Machine learning algorithms used in smart irrigation literature

Algorithm	Typical role	Advantages	Limitations
Decision Tree	Irrigation/no-irrigation rules and duration decisions	Interpretable, low computation cost	Can overfit simple datasets
Random Forest	Classification of irrigation need and robust tabular prediction	Robust, nonlinear, suitable for mixed variables	Less transparent than a single tree
SVM	Classification/regression for irrigation status	Effective in smaller datasets	Requires tuning and scaling
Gradient Boosting	Soil moisture and evapotranspiration prediction	High predictive performance	More complex tuning
ANN/LSTM	Time-series and nonlinear water demand prediction	Can model temporal dynamics	Requires larger datasets and careful validation
Reinforcement Learning	Optimization of irrigation policy over time	Can learn sequential decisions	Needs simulation environment and safety constraints

2.6 Simulation and what-if analysis

Simulation is a core capability of Digital Twins. In irrigation, what-if simulation can compare moisture evolution if irrigation is applied versus if it is skipped. This enables the user to evaluate a decision before applying it physically. Simulation can also help explain AI decisions because the user can see the expected trajectory rather than only a binary recommendation.

Agrio implements a Digital Twin simulation workspace. The user selects a zone, reads the live state, sets scenario inputs such as irrigation duration and projection horizon, runs the simulation, and compares outputs. The system displays predicted moisture, water balance, 24-hour and 48-hour projections, and a textual explanation. The simulation does not activate the pump; it only supports decision-making.

2.7 2.7 Human-in-the-loop Digital Twins

Human-in-the-loop design is important in agricultural Digital Twins. Farmers and agronomists possess domain knowledge that cannot be fully replaced by algorithms, especially in early prototypes. Human validation also improves safety when actuators are involved. In Agrio, recommendations can be validated by the user before becoming scheduled irrigation events. Manual pump control remains possible through the dashboard.

This design supports trust. Instead of hiding the logic behind automation, the platform shows recommendation status, confidence, duration, suggested time, and explanations. The user can then decide whether to accept the system output. In future versions, the system could learn from user feedback and execution results.

2.8 2.8 Digital Twin challenges in agriculture

Digital Twin adoption in agriculture faces major barriers: data heterogeneity, sensor calibration, limited connectivity, lack of standard architectures, high implementation cost, privacy and security concerns, limited interoperability, and difficulty of modeling biological systems. Recent reviews also note that many agricultural systems use the term Digital Twin loosely, even when they are closer to decision support tools or monitoring dashboards [5].

Agrio addresses some of these challenges by defining a complete technical architecture and by separating sensing, communication, backend processing, ML, simulation, user validation, and actuation. However, the project does not solve all challenges. It still requires long-term field validation, improved security, stronger agronomic calibration, and extension to multiple crops and zones.

2.9 2.9 Conclusion

This chapter presented the theoretical foundations of Digital Twins, IoT, and AI for smart irrigation. It clarified the difference between digital models, digital shadows, and Digital Twins, introduced the layered architecture required for cyber-physical synchronization, and explained the role of ML and what-if simulation in decision support. These concepts are used in the next chapters to analyze the state of the art and position the Agrio project.

Part II

State of the Art and Scientific Positioning

Chapter 3

Digital Twins in Agriculture and Irrigation: State of the Art

3.1 3.1 Review methodology

The state-of-the-art study was built from peer-reviewed papers and technical sources related to agricultural Digital Twins, smart irrigation, IoT-based monitoring, machine learning for irrigation, and Digital Twin water management. The reviewed papers include general reviews, systematic reviews, bibliometric reviews, and application papers. The objective was not only to summarize publications, but to identify patterns, limitations, and opportunities relevant to the proposed Agrio platform.

The review was organized around five questions: (1) How are Digital Twins defined and classified in agriculture? (2) Which enabling technologies are used? (3) How are Digital Twins applied to irrigation and water management? (4) Which machine learning approaches support irrigation decisions? (5) What gaps remain in current systems, and how does Agrio address some of them?

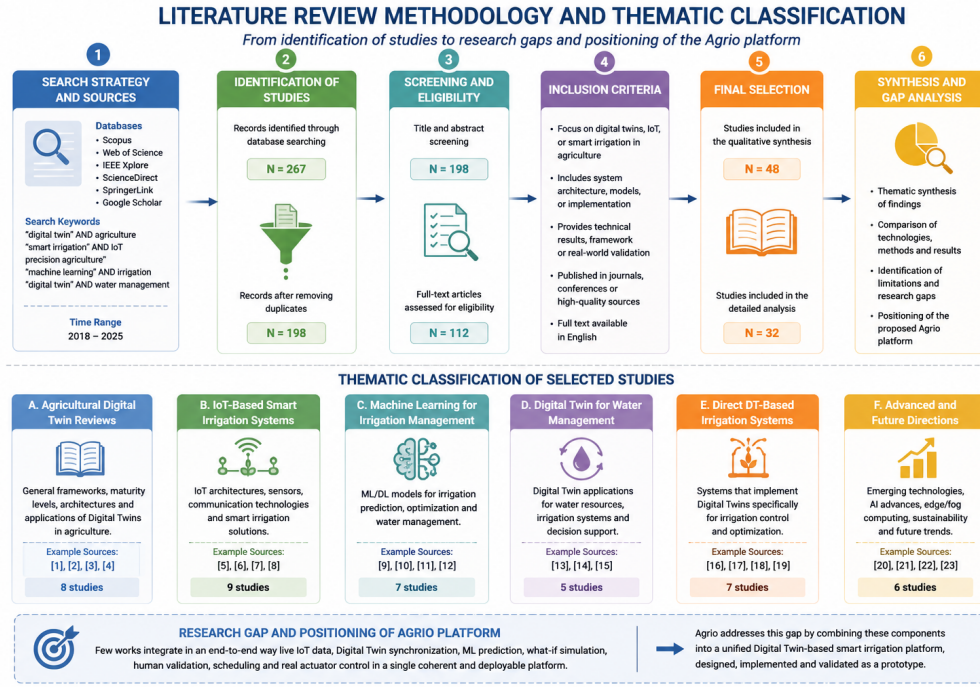


Figure 3.1: Literature review methodology and thematic classification. Source: Authors.

3.2 General agricultural Digital Twin reviews

Subeesh and Chauhan [1] provide a broad review of Agricultural Digital Twins. They describe ADTs as virtual representations that support decision-making, monitoring, simulation, and optimization in smart farming. Their review emphasizes enabling technologies such as IoT, AI, AR/XR, edge computing, federated learning, and sensors. A key point is that agriculture is still at an early stage of Digital Twin adoption, and that practical implementation is complicated by the stochastic nature of weather, soil, and biological processes.

Zhang et al. [2] present a comprehensive review of Digital Twin technology in agriculture. They discuss the distinction between Digital Model, Digital Shadow, and Digital Twin, and describe architecture layers such as perception, network, and application layers. This classification is particularly useful for the present thesis because it helps evaluate whether an agricultural system is only a monitoring platform or whether it includes feedback and decision capacity.

Gund et al. [3] use bibliometric analysis to map research trends in smart agricultural Digital Twins. They show that publications have grown rapidly since 2018 and that the main clusters include precision farming, IoT integration, AI, cyber-physical systems, controlled-environment agriculture, sustainability, and food systems. This confirms that Agrio is positioned inside an active and expanding research field.

Awais et al. [4] focus on precision agriculture through Digital Twins and smart farming technologies. They note that current research often remains isolated around specific applications such as irrigation, disease detection, or crop monitoring rather than providing unified, scalable frameworks. This observation supports the need for integrated platforms like Agrio that connect multiple components: sensing, backend, database, ML, simulation, dashboard, and actuation.

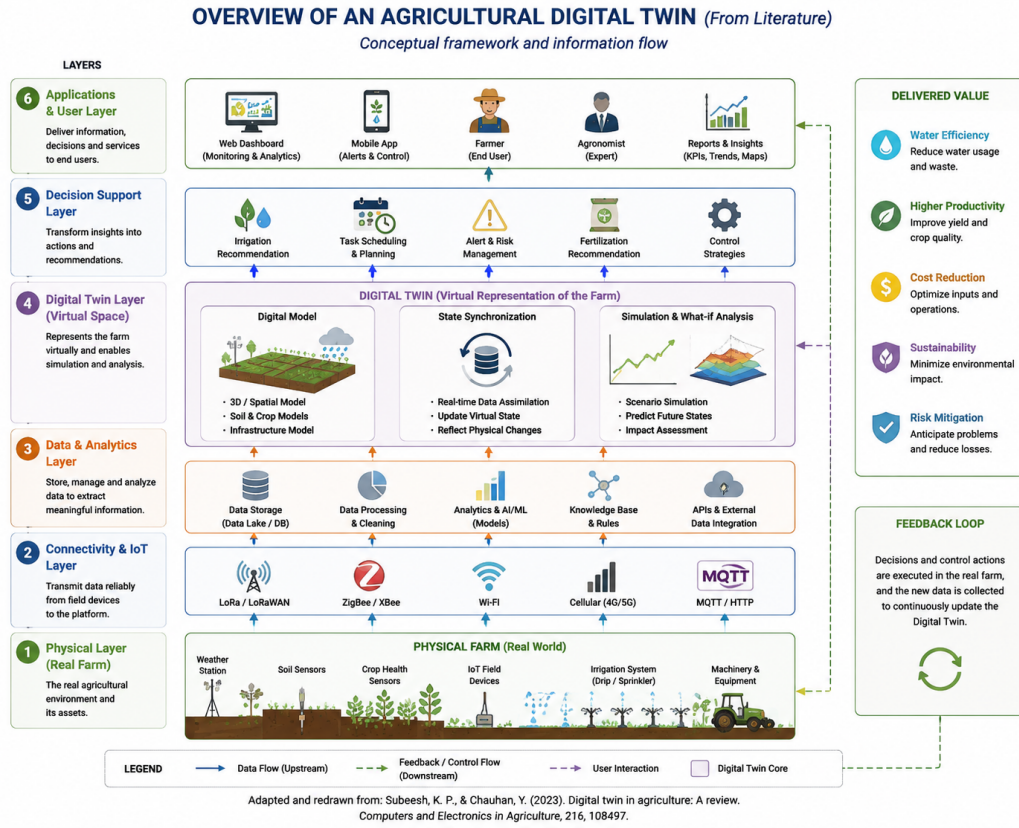


Figure 3.2: Example of agricultural Digital Twin overview from literature [insert or redraw based on Subeesh and Chauhan, Fig. 1]. Source: Adapted/redrawn from [1].

3.3 AI-enabled Digital Twins in agriculture

Tsaousidis et al. [5] provide a recent review focused on AI-enabled Digital Twins in agriculture. They emphasize that many works use the term Digital Twin loosely and sometimes describe systems that are closer to decision support tools than fully synchronized Digital Twins. This warning is important for the scientific rigor of this thesis. The proposed system must therefore explicitly demonstrate its Digital Twin properties: synchronization from sensor data, virtual state, prediction, simulation, recommendation, and feedback/control capability.

The same review highlights data integration as a major bottleneck, especially when combining satellite data, local sensor data, and simulation outputs. Agrio currently integrates local sensor data, weather/forecast data, and ML outputs, but it does not yet integrate satellite or UAV data. This limitation becomes a clear future direction.

3.4 Digital Twins for agricultural water management

Ahsen et al. [6] specifically review Digital Twins for sustainable agricultural water management. Their review identifies monitoring Digital Twins and predictive Digital Twins. Monitoring Digital Twins focus on real-time response and environmental control, while predictive Digital Twins support proactive management through forecasting and simulation. Agrio combines both aspects: it monitors current zone state and also simulates future moisture evolution.

Digital Twin water management research often focuses on irrigation, hydroponics, aquapon-

ics, and vertical farming. The main benefits include precise resource allocation, scenario planning, predictive maintenance, and optimized water usage. The main barriers include scalability, cost, data integration, and the difficulty of field deployment. These findings align with the scope and limitations of Agrio.

3.5 3.5 IoT-based smart irrigation reviews

García et al. [7] review IoT-based smart irrigation systems and analyze sensors, actuators, nodes, wireless technologies, and monitored parameters. They show that smart irrigation systems rely heavily on soil and weather monitoring. Soil moisture is a central variable, but weather parameters such as temperature, humidity, rainfall, wind, and radiation are also important. Their review supports the variable selection used in Agrio.

Obaideen et al. [8] provide an overview of smart irrigation using IoT and emphasize the relationship between irrigation systems and sustainable development. They describe smart irrigation as a system including data acquisition, irrigation control, wireless communication, data processing, and fault detection. Agrio follows these components but extends them with Digital Twin simulation and AI prediction.

Morchid et al. [10] systematically review IoT and ML applications in sustainable irrigation. Their PRISMA-based analysis highlights the benefits of real-time environmental information and ML-based decision-making, while also emphasizing barriers such as complexity, technical constraints, and limited adoption. Their work supports the decision to combine IoT, ML, and human validation rather than relying on fully autonomous decisions.

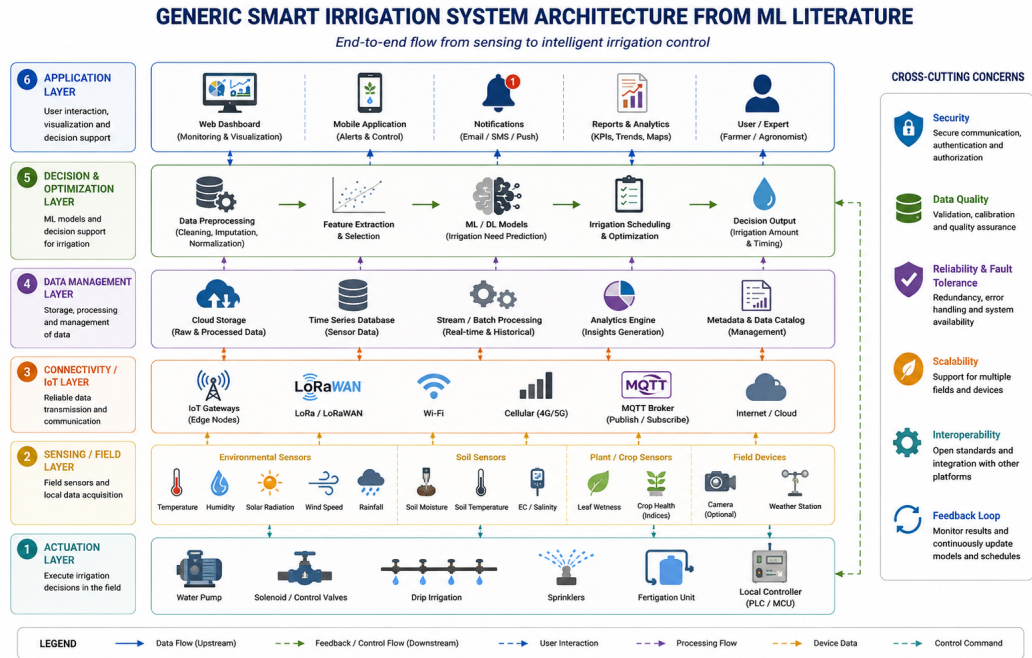


Figure 3.3: Generic smart irrigation system architecture from ML literature [insert/redraw based on Del-Coco et al., Fig. 1]. Source: Adapted/redrawn from [9].

3.6 3.6 Machine learning for smart irrigation

Del-Coco et al. [9] focus on machine learning for smart irrigation. They define a generic smart irrigation system as a combination of sensors, IoT technologies, algorithms, datasets, and decision services. They discuss existing ML strategies and note open issues such as long-term data processing and the lack of annotated datasets. This is important for Agrio because the Random Forest models must be presented as part of a prototype and not as universally validated agronomic models.

The reviewed ML literature includes decision trees, Random Forests, SVM, gradient boosting, neural networks, and LSTM models. Random Forests are used in several smart irrigation studies for binary irrigation decisions or pump activation/deactivation. This supports the selection of Random Forest in Agrio, especially because the project uses structured tabular features derived from sensor and environmental data.

One challenge in ML irrigation is interpretability. Farmers may not trust a black-box output if it simply says irrigate or do not irrigate. Agrio addresses this partially by showing confidence, recommended action, suggested time, duration, and explanation in the dashboard. The Digital Twin simulation also provides visual and numerical evidence of expected moisture evolution.

3.7 3.7 Directly related Digital Twin irrigation studies

Manocha et al. [15]: IoT-Digital Twin-inspired smart irrigation for optimal water utilization. Uses IoT sensors, virtual counterparts, irrigation requirement calculation, and simulation. Very close to Agrio conceptually; useful for comparing Digital Twin maturity and architecture.

Tancredi et al. [16]: Digital Twin enhanced with ML algorithms for irrigation management using sensor data. Uses ML and sensor data for irrigation ON/OFF decisions, with evaluation metrics such as accuracy, precision, recall, F1, and confusion matrix. Very close to Agrio ML layer.

Qin et al. [17]: Digital Twin-enabled intelligent irrigation-drainage system for water-salt management. Advanced system for precision water-salt management, useful as an example of deeper agronomic modeling and field-oriented Digital Twin irrigation.

Millán et al. [13]: Digital Twin for water-efficient management in a processing tomato commercial farm. Important real-field validation on a 15 ha tomato farm using soil moisture, climate data, simulation models, and daily scheduling. Useful to show what Agrio should target in future field deployment.

Bellvert et al. [14]: Sentinel-2 variables assimilated into a Digital Twin for vineyard irrigation scheduling. Demonstrates remote sensing integration, automated scheduling, and Digital Twin water balance simulation. Useful for future extension with satellite/UAV data.

El Hachimi et al. [19]: Digital Twins and deep reinforcement learning for irrigation water distribution. Advanced optimization direction beyond Agrio, useful for future work.

Liu et al. [18]: Cotton canopy Digital Twin under water stress. Shows plant-water stress modeling and crop visualization aspects, useful for discussing advanced crop-specific twins.

Table 3.1: Key literature themes used in this thesis

Theme	Representative sources	Main insight for Agrio
Agricultural Digital Twins	[1], [2], [3], [4], [5]	DTs require virtual representation, synchronization, simulation, prediction, and feedback; agriculture remains an emerging application domain.
Water-management DTs	[6], [13], [14], [15], [17]	Irrigation DTs can support daily scheduling, water balance simulation, and scenario-based management.
IoT smart irrigation	[7], [8], [10], [11], [12]	Smart irrigation systems depend on sensing, wireless communication, cloud processing, and actuator control.
ML irrigation decisions	[9], [10], [16], [21]	ML can classify irrigation need and suggest timing, but requires reliable datasets and clear evaluation.
Advanced future directions	[14], [18], [19]	Remote sensing, water-stress modeling, and reinforcement learning represent future improvements for Agrio.

3.8 Research gaps identified in the literature

The reviewed literature reveals several gaps. First, many agricultural Digital Twin works remain conceptual or focused on monitoring rather than full decision and actuation loops. Second, many smart irrigation systems use threshold rules or simple dashboards without simulation. Third, ML irrigation papers often focus on model accuracy but do not integrate the model into a complete deployed system. Fourth, systems with strong agronomic validation may not provide full software architecture details. Fifth, very few academic prototypes clearly combine physical sensors, MQTT ingestion, database persistence, ML inference, Digital Twin simulation, human validation, web/mobile interfaces, scheduling, and actuator feedback in one coherent workflow.

Agrio addresses part of this gap by implementing an integrated platform. It does not claim to solve every agronomic challenge, but it demonstrates how a Digital Twin irrigation system can be architected and deployed from the physical prototype to the dashboard and pump. Its novelty is therefore mainly architectural and integrative: combining multiple layers that are often studied separately.

3.9 Conclusion

This chapter reviewed the state of the art of Digital Twins in agriculture, IoT-based smart irrigation, ML irrigation decision-making, and direct Digital Twin irrigation applications. The literature shows strong interest in DTs but also highlights fragmentation, limited field validation, and integration challenges. The next chapter compares existing works more systematically and positions Agrio with respect to scientific and engineering criteria.

Chapter 4

Comparative Analysis and Scientific Positioning

4.1 Comparison criteria

To position Agrio scientifically, existing works are compared according to criteria that reflect Digital Twin maturity and smart irrigation functionality. The criteria include application domain, sensing layer, communication layer, data storage, machine learning, simulation, recommendation, user validation, actuation, deployment, and validation context. These criteria allow a fair comparison between research papers that may focus on different aspects of the problem.

The purpose of this comparison is not to claim that Agrio is superior to field-validated agricultural systems. Instead, it identifies what the project contributes as an integrated engineering prototype and where it remains limited compared with advanced research systems.

Table 4.1: Comparison criteria for Digital Twin irrigation systems

Criterion	Question evaluated
Physical sensing	Does the system receive real sensor data?
Communication	Which technology transmits data?
Data management	Is data stored and structured?
Virtual state	Is there a synchronized virtual representation?
ML/AI	Is prediction or optimization used?
Simulation	Are what-if or future scenarios simulated?
Recommendation	Does the system generate decisions?
Human validation	Can the user validate/override decisions?
Actuation	Can the system control pump/valves?
Deployment	Is there a reproducible implementation?
Validation	Is it tested in lab, prototype, or real farm?

4.2 4.2 Comparative matrix

Table 4.2: Scientific comparison of selected works with Agrio

Study	Focus	Main components	DT maturity	Position relative to Agrio
García et al. [7]	IoT smart irrigation review	Sensors, nodes, WSN/IoT, actuators	No specific DT; broad review	Supports Agrio sensing and communication background
Obaideen et al. [8]	IoT smart irrigation overview	Soil/weather monitoring, control, SDG link	No specific DT implementation	Supports sustainability and smart irrigation motivation
Del-Coco et al. [9]	ML smart irrigation review	Physical, processing, dataset, decision layers	No specific prototype	Supports Agrio ML and layered architecture
Kodali & Sarjerao [11]	Low-cost MQTT irrigation	Soil moisture, ESP8266, MQTT, pump status	IoT control, no DT simulation	Similar MQTT/pump logic but simpler than Agrio
Balaceanu et al. [12]	Libelium precision agriculture monitoring	Libelium station, temperature, soil humidity, pressure, radiation	Monitoring platform	Supports Agrio hardware choice
Manocha et al. [15]	IoT-DT smart irrigation	IoT sensors, simulation, irrigation suggestions	Integrated DT framework	Close to Agrio; useful benchmark
Tancredi et al. [16]	DT + ML irrigation	Sensor data, ML classification	DT enhanced with ML	Close ML comparison; Agrio adds full dashboard/scheduler/control
Qin et al. [17]	DT irrigation-drainage	IoT, LSTM, water-salt management	Advanced irrigation DT	Agronomically stronger; Agrio is broader software prototype
Millán et al. [13]	Tomato farm DT	Soil moisture, climate, simulation, daily scheduling	Commercial field DT	Field validation stronger than Agrio; future target
Bellvert et al. [14]	Vineyard DT with Sentinel-2	Remote sensing, fAPAR, SWB simulation, scheduling	Commercial vineyard DT	Shows future remote sensing integration
Agrio	This thesis	Libelium sensors, XBee, Raspberry Pi, MQTT, FastAPI, PostgreSQL, RF ML, simulation, dashboard, WeMos pump	Human-supervised operational prototype DT	Integrative architecture: sensing-to-actuation pipeline tested in controlled prototype

4.3 Positioning of Agrio by Digital Twin maturity

Agrio can be positioned between a Digital Shadow and a fully autonomous Digital Twin. It has a real sensing layer and updates the virtual zone state using live MQTT data, which goes beyond an offline Digital Model. It also generates decisions and can control the pump through MQTT commands, which goes beyond a one-way Digital Shadow. However, because autonomous actuation is intentionally limited and the prototype is not yet validated in a real farm over a full growing season, the system should be described as a human-supervised operational Digital Twin prototype.

This positioning is academically honest and defensible. It acknowledges that field-validated systems such as Irri_DesK applied to commercial tomato or vineyard plots have stronger agronomic validation. At the same time, it highlights Agrio's engineering completeness: backend architecture, database model, ML service, simulation interface, web/mobile dashboard, scheduling, and actuator feedback.

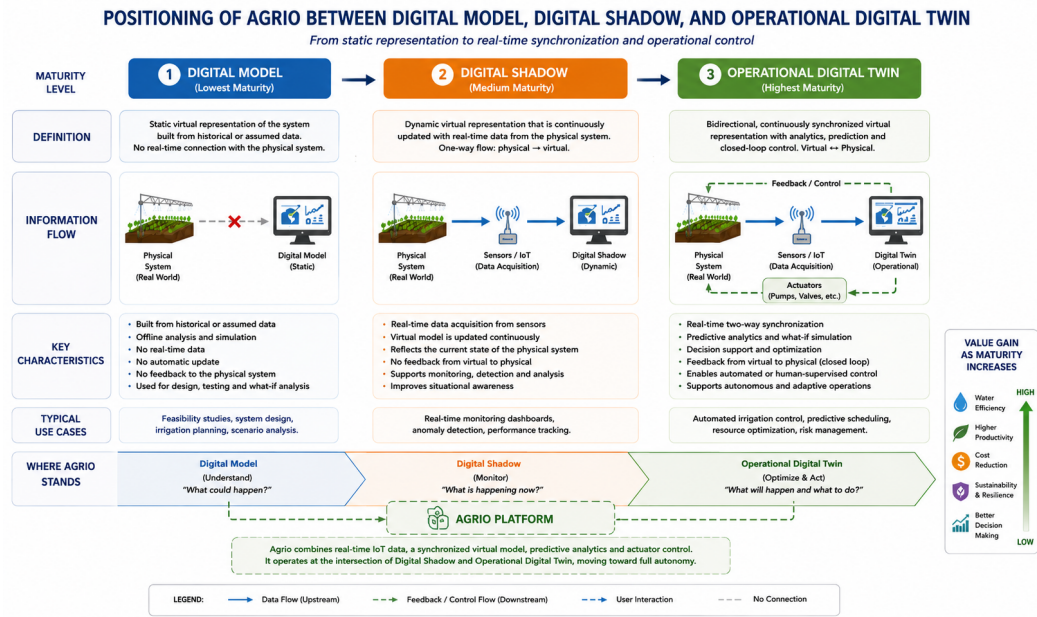


Figure 4.1: Positioning of Agrio between Digital Model, Digital Shadow, and operational Digital Twin. Source: Authors.

4.4 Novelty of the proposed project

Table 4.3: Novelty and contribution of Agrio

Contribution	Explanation
End-to-end integration	Many works focus on one layer; Agrio connects field sensing, RF communication, gateway, MQTT, backend, database, ML, simulation, dashboard, scheduling, and pump control.
Physical prototype	The system is not only conceptual; it uses a controlled prototype with Libelium kit, XBee, Raspberry Pi, WeMos, and water pump.

Contribution	Explanation
Human-supervised Digital Twin	Recommendations are not blindly executed. The user can validate, schedule, or manually control the pump.
What-if simulation	The platform compares no-irrigation, with-irrigation, and ML-selected paths before physical action.
Deployment-oriented engineering	Docker Compose and separated services make the prototype reproducible and easier to demonstrate.
Traceable database model	The schema stores measurements, states, history, predictions, simulations, recommendations, commands, events, and feedback.

4.5 Weaknesses and limitations identified from the comparison

The comparison also reveals weaknesses. Agrio is currently validated in a controlled prototype environment, not in a full commercial farm. The ML models rely on adapted datasets and project data, but not on multi-season field datasets. Agronomic calibration of crop coefficients, soil hydraulic parameters, and water balance models is still limited. The system does not yet include remote sensing, UAV imagery, salinity management, reinforcement learning, or optimization of multi-zone water distribution.

Another limitation is security. A production irrigation platform would require robust authentication, authorization, encrypted device provisioning, secure MQTT credentials, audit logs, and protection against unauthorized pump commands. The current project prioritizes functional integration for academic validation, while production security remains future work.

These limitations do not reduce the value of the prototype; rather, they define a realistic boundary. The thesis should present Agrio as a strong final-year engineering prototype and research-informed Digital Twin platform, not as a commercially validated agricultural product.

4.6 Conclusion

This chapter positioned Agrio among existing smart irrigation and Digital Twin systems. The contribution lies in integrating multiple layers into an operational prototype and demonstrating the Digital Twin loop from sensing to simulation and control. The comparison also clarified the limitations of the project, especially the need for real field validation, stronger agronomic calibration, and production-level security. The next part presents the requirements and architecture of the proposed system.

Part III

Requirements and Architecture

Chapter 5

Requirements Specification and Design

5.1 Introduction

This chapter specifies the functional and non-functional requirements of the Agrio platform. Requirements are derived from the project objectives, literature gaps, and real implementation artifacts such as the backend routes, database schema, MQTT payload catalog, dashboard pages, and actuator workflow. The chapter also defines the actors and the main use cases of the system.

5.2 System objectives

The main objective of Agrio is to support intelligent irrigation decisions through a Digital Twin of a monitored agricultural zone. The system must receive sensor data from the physical prototype, store it, update the virtual state, predict irrigation need, simulate scenarios, generate recommendations, display results to the user, and control the pump when a decision is validated or manually triggered.

The system must also be demonstrable and maintainable. This means that it should expose clear API endpoints, store data in a structured schema, separate the ML service from the backend, provide a dashboard for testing and operation, and support deployment through Docker Compose.

5.3 Actors

The main human actors are the farmer/operator and the administrator/agronomist. The farmer monitors the dashboard, checks zone conditions, reviews recommendations, validates decisions, schedules irrigation, and manually controls the pump if needed. The administrator or agronomist manages farms, plots, zones, sensors, soil profiles, crops, and system configuration. External technical actors include the MQTT sensor station, WeMos actuator, ML service, Open-Meteo API, and PostgreSQL database.

5.4 5.4 Functional requirements

Functional requirements describe what the system must do. Agrio must ingest MQTT sensor payloads, normalize fields, map devices to zones, store readings, maintain weather records, evaluate sensor health, update Digital Twin zone states, call the ML service, store prediction results, run what-if simulations, generate recommendations, validate decisions, create schedules, publish actuator commands, receive WeMos status, and expose dashboard data through APIs.

5.5 5.5 Non-functional requirements

Non-functional requirements describe quality constraints. The system must be modular, maintainable, reproducible, and observable. It must support near-real-time dashboard updates, reliable message parsing, safe actuation, clear separation of simulation and physical control, and extensibility toward more zones and crops. The prototype also requires usability because the dashboard is central to decision support.

5.6 5.6 Constraints

The current system is constrained by the laboratory prototype, limited long-term field data, and available hardware. It is designed for controlled demonstration rather than immediate production deployment. The platform currently prioritizes functionality, integration, and traceability over advanced cybersecurity and large-scale distributed deployment. These constraints are documented to maintain academic transparency.

Table 5.1: Functional requirements

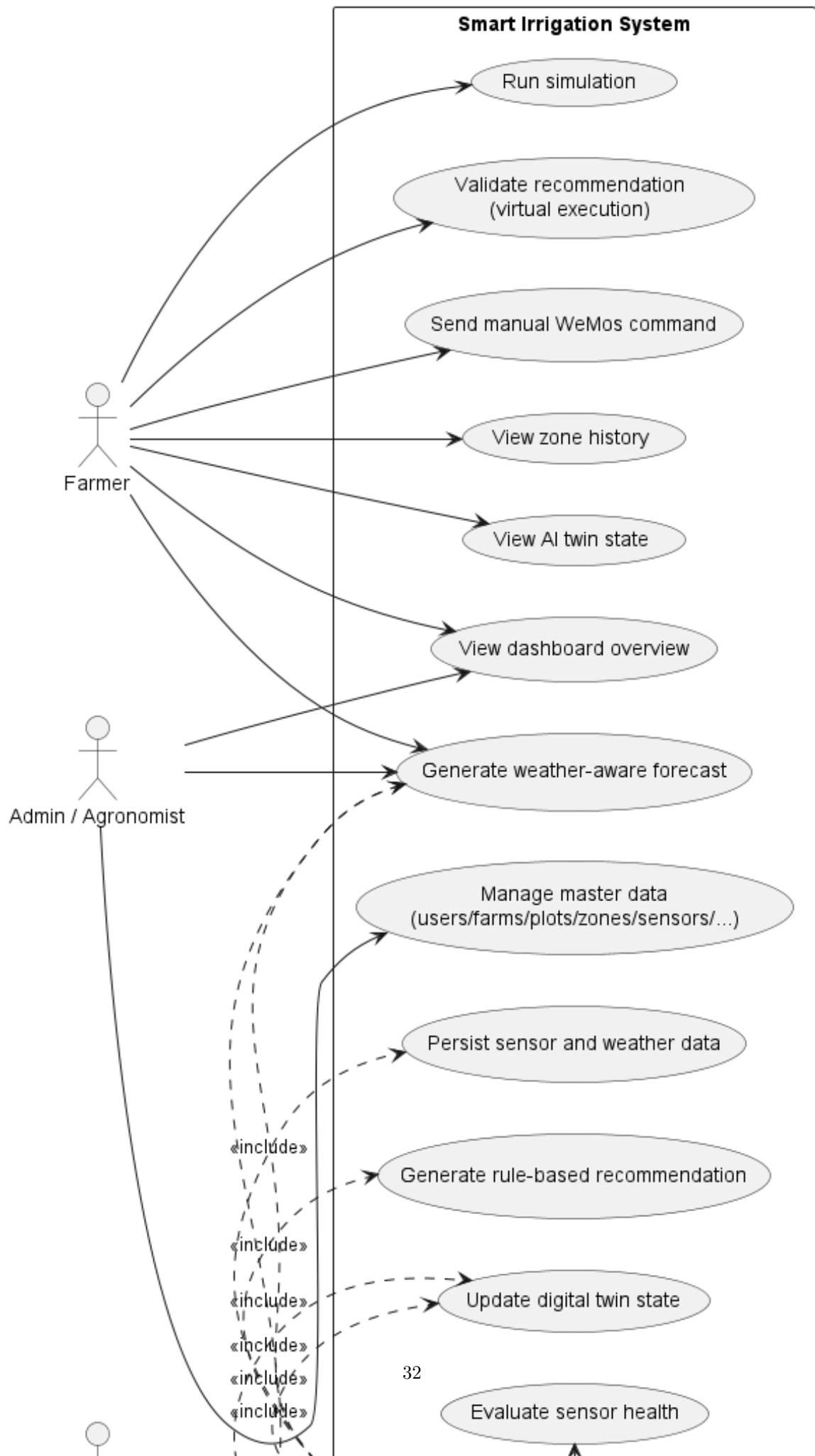
ID	Requirement	Description
FR1	Ingest sensor payloads	Receive JSON messages from sensors/data through MQTT.
FR2	Normalize measurements	Parse aliases, timestamps, moisture fractions, light suffixes, and wind direction values.
FR3	Persist readings	Store sensor_readings and weather_records in PostgreSQL.
FR4	Update Digital Twin	Build and update zone_states and zone_state_history.
FR5	Predict irrigation need	Call ML service and store prediction_results.
FR6	Run what-if simulation	Compare no-irrigation and with-irrigation scenarios.
FR7	Generate recommendation	Create pending or validated recommendations.
FR8	Schedule irrigation	Create irrigation_events with start time and duration.
FR9	Control actuator	Send ON_D5/OFF_D5 commands to WeMos through MQTT.
FR10	Display dashboard	Provide live monitoring, prediction, simulation, forecast, schedules, and decisions.

Table 5.2: Non-functional requirements

ID	Requirement	Design implication
NFR1	Modularity	Separate backend, ML service, frontend, scheduler, and database.
NFR2	Traceability	Persist raw readings, states, predictions, recommendations, commands, and feedback.
NFR3	Safety	Separate simulation from physical actuation and require validation.
NFR4	Usability	Provide readable dashboard cards, charts, explanations, and manual controls.
NFR5	Reproducibility	Use Docker Compose and environment variables.
NFR6	Extensibility	Use farms, plots, zones, sensors, and device mapping for future scale.
NFR7	Observability	Expose API docs, status endpoints, logs, and database records.
NFR8	Performance	Support near-real-time polling and fast ML inference for prototype operation.

5.7 5.7 Use-case model

The use-case diagram represents the interaction between the farmer, administrator, IoT sensor station, WeMos actuator, ML service, and Open-Meteo API. It includes monitoring, simulation, recommendation validation, manual command, forecast generation, data ingestion, sensor health evaluation, and ML prediction.



5.8 5.8 Activity workflow

The activity workflow begins when the backend receives an MQTT sensor payload. The system validates and normalizes the payload, resolves the zone, creates missing sensor mappings if needed, inserts weather records and sensor readings, evaluates sensor health, updates the zone state, calls the ML service, stores predictions, appends state history, and exposes the updated state to the dashboard. If irrigation is required, the system waits for user confirmation. Validation can create a recommendation record and irrigation event, while manual device commands publish MQTT commands to the WeMos actuator.

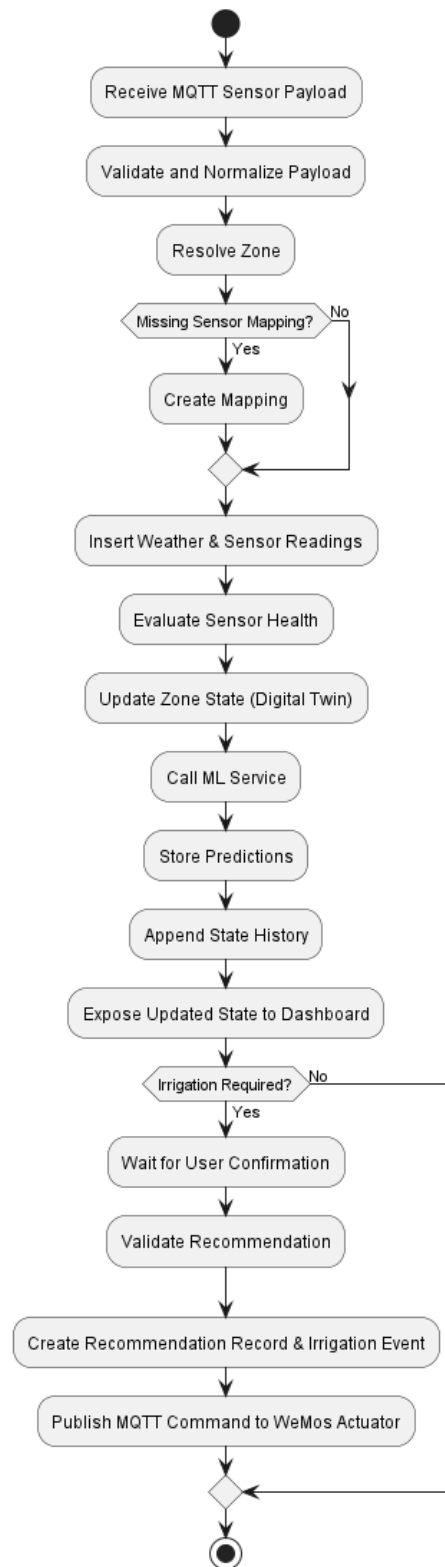


Figure 5.2: Activity diagram of sensor ingestion, Digital Twin update, AI decision-making, and pump control. Source: Authors.

5.9 5.9 Domain class diagrams

The full domain model is too large to be readable as one figure. Therefore, it is split into four diagrams: agricultural domain, sensor/weather/forecast domain, Digital Twin/prediction/simulation domain, and decision/control domain. The complete class diagram is kept in the appendix.

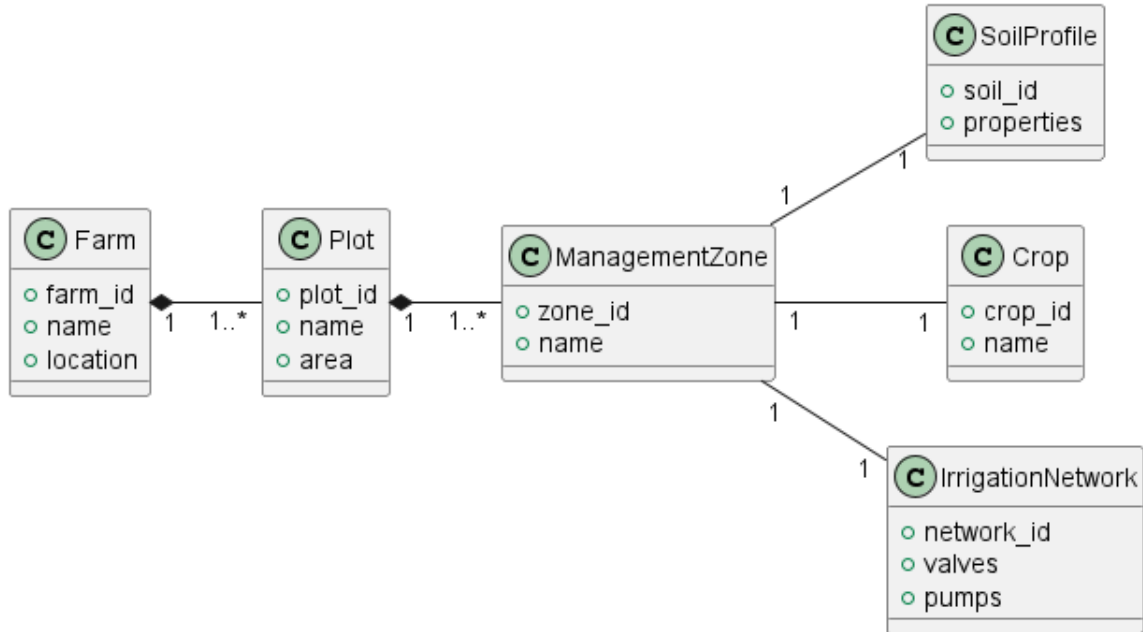


Figure 5.3: Agricultural domain class diagram representing farms, plots, zones, soil profiles, crops, and irrigation networks. Source: Authors.

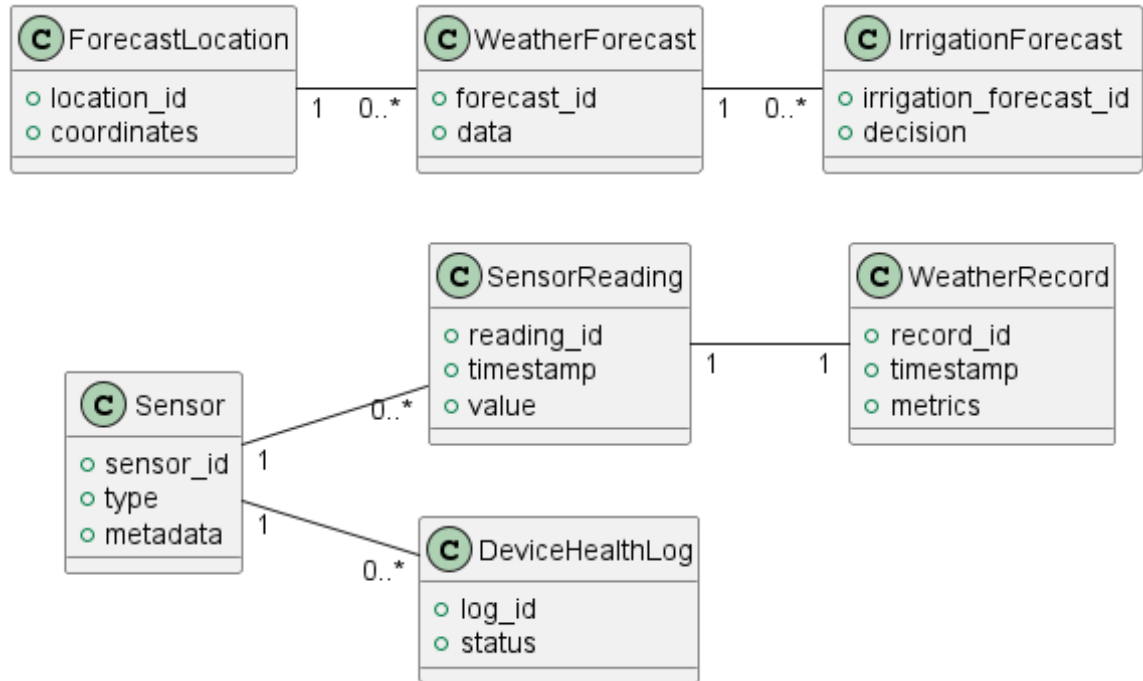


Figure 5.4: Sensor, weather, and forecasting data model used by the smart irrigation platform. Source: Authors.

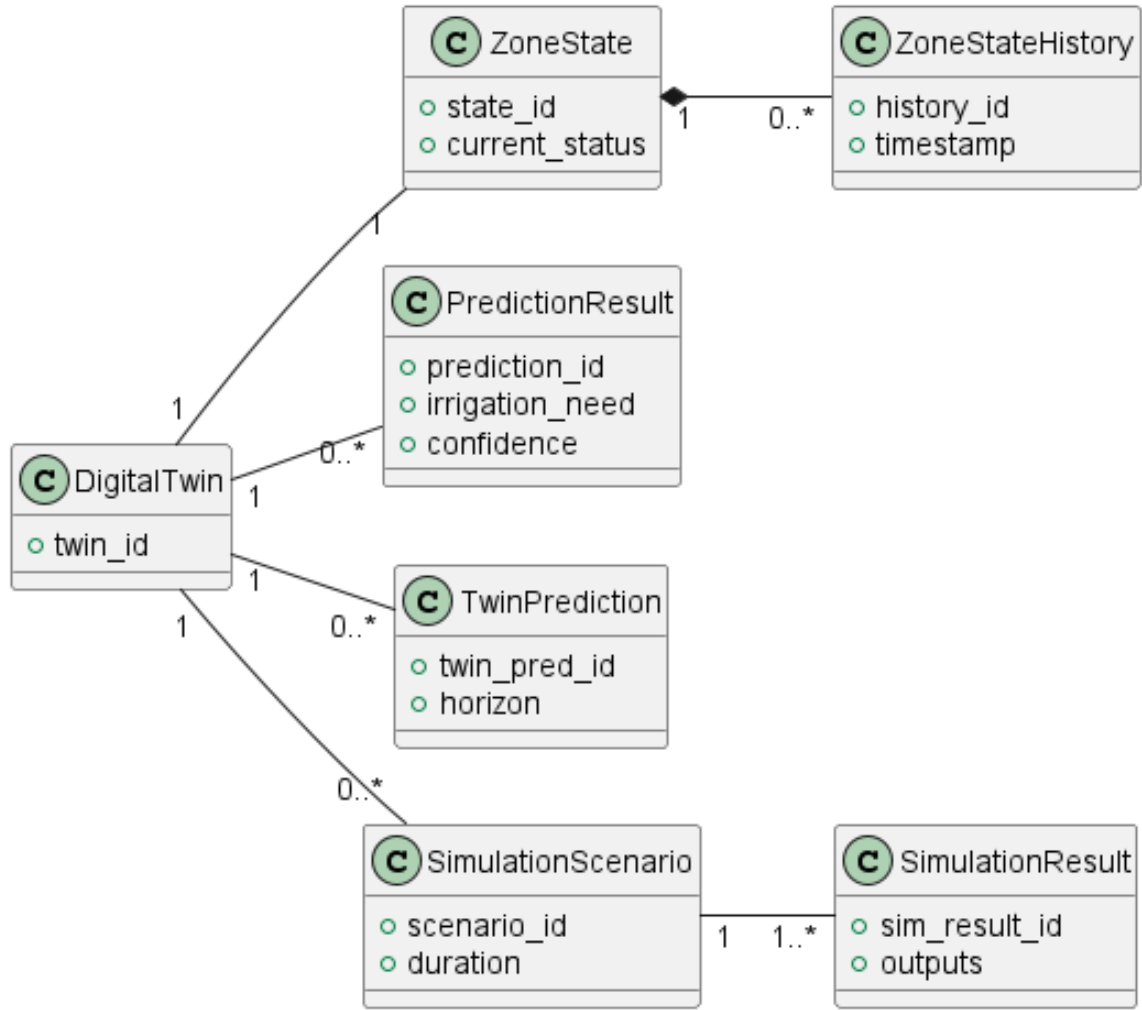


Figure 5.5: Digital Twin, prediction, and simulation data model. Source: Authors.

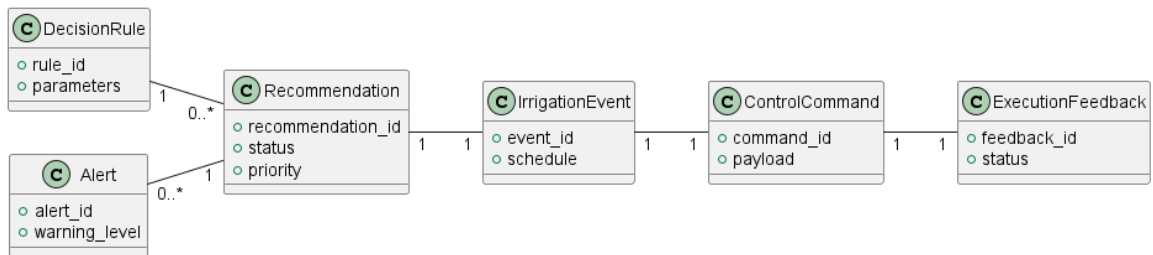


Figure 5.6: Decision, recommendation, scheduling, command, and feedback data model. Source: Authors.

5.10 Conclusion

This chapter defined the requirements and design foundations of Agrio. It described the actors, functional requirements, non-functional requirements, constraints, use cases, workflow, and domain class diagrams. The next chapter presents the global architecture that satisfies these requirements.

Chapter 6

Global Architecture of the Proposed Digital Twin

6.1 Introduction

This chapter presents the global architecture of the Agrio Digital Twin smart irrigation platform. The architecture is designed as a cyber-physical loop: sense, transmit, store, synchronize, predict, simulate, recommend, validate, actuate, and record feedback. Each layer has a clear role and communicates with other layers through defined interfaces.

6.2 Architectural overview

The architecture connects Libelium Smart Agriculture sensors, XBee radio-frequency communication, Raspberry Pi gateway processing, HiveMQ/MQTT messaging, FastAPI backend services, PostgreSQL storage, a Random Forest ML service, a Digital Twin engine, recommendation and scheduler services, a Next.js dashboard, a Flutter mobile application, a WeMos actuator, and a water pump. This organization ensures that the physical prototype and its virtual representation remain connected.

Insert Figure 6.1 here: Global overview of the Agrio Digital Twin smart irrigation architecture.

Figure 6.1: Global overview of the Agrio Digital Twin smart irrigation architecture. Source: Authors.

6.3 Physical prototype layer

The physical layer is a controlled experimental environment with plants, irrigation lines, sensing equipment, and a water pump. It reproduces the essential elements of smart irrigation at laboratory scale. It is not a commercial farm, but it provides a realistic environment for validating data acquisition, communication, virtual state update, simulation, and actuation.

6.4 6.4 Sensing layer

The sensing layer uses the Libelium Smart Agriculture kit. Measurements include soil moisture, soil temperature, air temperature, humidity, pressure, rainfall, wind speed, wind direction, leaf wetness, and light intensity. These variables are published as structured JSON payloads after gateway processing.

6.5 6.5 Radio-frequency and gateway layer

XBee radio-frequency communication transfers field data to the gateway. The Raspberry Pi receives the data through the XBee gateway, decodes or decrypts the payload when required, prepares a standardized JSON message, and publishes it to HiveMQ using MQTT. This layer transforms physical measurements into cloud-consumable IoT messages.

Insert Figure 6.2 here: Physical and communication architecture of the controlled smart irrigation prototype.

Figure 6.2: Physical and communication architecture of the controlled smart irrigation prototype. Source: Authors.

6.6 6.6 MQTT communication layer

MQTT organizes communication around topics. The sensors/data topic is used for gateway-to-backend sensor payloads. The smart_irrigation/wemos1/status topic is used for WeMos-to-backend status feedback. The smart_irrigation/wemos1/cmd topic is used for backend-to-WeMos commands. This bidirectional messaging supports both physical-to-digital synchronization and digital-to-physical control.

Insert Figure 6.3 here: MQTT topic architecture for sensor ingestion, actuator commands, and WeMos status feedback.

Figure 6.3: MQTT topic architecture for sensor ingestion, actuator commands, and WeMos status feedback. Source: Authors.

6.7 6.7 Backend layer

The FastAPI backend is the central orchestrator. It exposes REST endpoints, subscribes to MQTT topics, parses and persists incoming data, updates the Digital Twin state, calls ML inference, generates recommendations, manages forecasts, schedules irrigation events, and publishes actuator commands.

Insert Figure 6.4 here: Internal backend architecture of the FastAPI-based Digital Twin platform.

Figure 6.4: Internal backend architecture of the FastAPI-based Digital Twin platform. Source: Authors.

6.8 6.8 Database layer

PostgreSQL stores the agricultural structure, sensor readings, weather records, forecasts, Digital Twin states, predictions, simulations, recommendations, alerts, irrigation events, control commands, and execution feedback. The schema is not limited to raw data; it supports the complete Digital Twin lifecycle.

6.9 6.9 Digital Twin layer

The Digital Twin layer builds the virtual state of each management zone. It includes current moisture, soil temperature, water balance, stress level, irrigation need, recommended action, confidence, synchronization timestamp, and pump state. It also stores history and generates future projections.

Insert Figure 6.5 here: Digital Twin processing architecture from live sensor data to simulation and recommendation.

Figure 6.5: Digital Twin processing architecture from live sensor data to simulation and recommendation. Source: Authors.

6.10 6.10 Machine learning layer

The ML service is separated from the backend and exposes prediction endpoints. It loads trained Random Forest models and returns irrigation need, suggested hour, confidence values, and recommended action. Separation improves maintainability and allows future model replacement.

6.11 6.11 Recommendation and validation layer

The recommendation engine transforms predictions and twin states into user-oriented decisions. Recommendations can be pending, validated, or converted into scheduled irrigation events. Human validation is intentionally preserved to improve safety and trust.

6.12 6.12 Actuator and feedback layer

The WeMos actuator receives commands such as ON_D5 and OFF_D5. It switches pump state and publishes status feedback. This closes the operational loop between digital decisions and physical irrigation.

6.13 6.13 Deployment architecture

Docker Compose defines the runtime environment, including PostgreSQL, backend, ML service, frontend, scheduler, ngrok, and optional workers. This makes the platform reproducible for demonstration and testing.

Insert Figure 6.6 here: Docker Compose deployment architecture of the Agrio platform.

Figure 6.6: Docker Compose deployment architecture of the Agrio platform. Source: Authors.

Insert Figure 6.7 here: Detailed technical architecture of the implemented Digital Twin smart irrigation platform.

Figure 6.7: Detailed technical architecture of the implemented Digital Twin smart irrigation platform. Source: Authors.

6.14 6.14 Discussion

The architecture is modular and extensible. It supports real-time ingestion, virtual synchronization, ML prediction, simulation, user validation, and actuator control. Its main limitations are controlled-environment validation, limited security hardening, and the absence of multi-season field data.

6.15 6.15 Conclusion

This chapter showed how the proposed architecture implements the Digital Twin concept in the context of smart irrigation. The following chapter details backend, database, and IoT ingestion implementation.

Table 6.1: Architecture layers and implementation choices

Layer	Implementation in Agrio	Role
Physical	Controlled prototype, plants, pump, irrigation line	Real environment to monitor and control
Sensing	Libelium Smart Agriculture kit	Collects soil, weather, and environmental variables
RF/Gateway	XBee + Raspberry Pi	Receives, decodes, and forwards data
Messaging	HiveMQ/MQTT	Publishes sensor data and device commands/status
Backend	FastAPI	Orchestrates ingestion, APIs, pipeline, decisions
Database	PostgreSQL	Stores complete Digital Twin lifecycle data
ML	Random Forest service	Predicts irrigation need and suggested hour
Digital Twin	Zone states, history, simulation	Represents and projects virtual zone behavior
Interface	Next.js + Flutter	Monitoring, simulation, forecast, schedules, commands
Actuation	WeMos + pump	Executes validated irrigation commands

Part IV

Implementation

Chapter 7

Backend, Database, and IoT Data Ingestion

7.1 7.1 Introduction

This chapter describes the implementation of the backend, database, and IoT ingestion pipeline. It explains how MQTT messages become database records, Digital Twin states, ML predictions, and dashboard updates.

7.2 7.2 FastAPI backend organization

The backend is implemented with FastAPI. It exposes REST endpoints under the `/api/v1` prefix, provides automatic OpenAPI documentation, configures CORS for frontend/mobile access, initializes services, and starts the MQTT subscriber when the system is configured for MQTT data source mode.

7.3 7.3 API modules

The API is organized around users, farms, plots, soil profiles, crops, zones, sensors, weather records, irrigation events, Digital Twins, predictions, recommendations, control commands, simulations, dashboard aggregation, device control, ML proxy, forecasting, system status, and debugging. This organization reflects the functional separation of the platform.

7.4 7.4 MQTT payload structure

Sensor messages are published on `sensors/data` as JSON. A typical message contains `device_id`, `timestamp`, `soil_moisture`, `soil_temperature`, `temperature`, `humidity`, `pressure`, `leaf_wetness`, `light_intensity`, `rainfall_current`, `rainfall_hour`, `rainfall_day`, `wind_direction`, and `wind_speed`. WeMos status messages contain `device_id` and digital pin states such as `D5` and `D3`.

Insert Figure 7.6 here: Example MQTT payloads used for sensor ingestion, WeMos status feedback, and actuator commands.

Figure 7.1: Example MQTT payloads used for sensor ingestion, WeMos status feedback, and actuator commands. Source: Authors.

7.5 7.5 Parser and normalization

The parser validates JSON, accepts aliases, converts soil moisture fractions into percentages, parses timestamps, normalizes numeric values, handles light-intensity suffixes, and normalizes wind direction. This step is essential because IoT messages can be inconsistent and sensor formats may differ.

7.6 7.6 Device mapping and persistence

After parsing, the backend resolves the device and management zone. It creates sensor mappings if required, stores `sensor_readings`, stores `weather_records` when environment fields exist, and updates the runtime MQTT status store. The source field preserves the origin of the record, for example `mqtt:device_id`.

7.7 7.7 Real-time pipeline

The real-time pipeline evaluates sensor health, updates zone state, calls the ML service, stores prediction results, appends `zone_state_history`, and generates recommendations. This pipeline converts raw measurements into actionable Digital Twin data.

7.8 7.8 PostgreSQL schema

The PostgreSQL schema contains tables for users, farms, plots, zones, soil profiles, crops, irrigation networks, sensors, readings, weather records, device health, forecasts, digital twins, zone states, predictions, simulations, recommendations, alerts, commands, events, and feedback. This structure supports traceability from sensor data to action.

Insert Figure 7.5a here: ERD of the agricultural structure tables.

Figure 7.2: ERD of the agricultural structure tables. Source: Authors.

Insert Figure 7.5b here: ERD of the sensor, weather, and forecasting tables.

Figure 7.3: ERD of the sensor, weather, and forecasting tables. Source: Authors.

Insert Figure 7.5c here: ERD of the Digital Twin, prediction, and simulation tables.

Figure 7.4: ERD of the Digital Twin, prediction, and simulation tables. Source: Authors.

Insert Figure 7.5d here: ERD of the recommendation, scheduling, command, and feedback tables.

Figure 7.5: ERD of the recommendation, scheduling, command, and feedback tables. Source: Authors.

7.9 7.9 Runtime sequence

The runtime sequence demonstrates how a sensor message travels from the sensor station to the MQTT broker, backend subscriber, parser, database, real-time pipeline, ML service, dashboard, device-control service, WeMos, and status feedback.

Insert Figure 7.1 here: Runtime sequence of MQTT data ingestion, ML prediction, dashboard polling, and WeMos actuator command.

Figure 7.6: Runtime sequence of MQTT data ingestion, ML prediction, dashboard polling, and WeMos actuator command. Source: Authors.

7.10 7.10 Error handling and fallback

If payload validation fails, the ingestion is marked as ignored or error. If the ML service is unavailable, the system can build a fallback rule-based prediction and store a fallback twin state. This prevents complete workflow failure when the ML service is temporarily unavailable.

7.11 7.11 Conclusion

The backend and data ingestion implementation prove that Agrio is not only a frontend demonstration. It contains a real data pipeline from MQTT ingestion to database persistence, Digital Twin synchronization, prediction, recommendation, and control.

Insert Figure 7.2 here: Live MQTT payloads received through HiveMQ for sensor data and WeMos status.

Figure 7.7: Live MQTT payloads received through HiveMQ for sensor data and WeMos status. Source: Authors.

Insert Figure 7.3 here: FastAPI OpenAPI documentation exposing the REST endpoints of the backend.

Figure 7.8: FastAPI OpenAPI documentation exposing the REST endpoints of the backend. Source: Authors.

Insert Figure 7.4 here: PostgreSQL stored sensor readings persisted from MQTT-originated measurements.

Figure 7.9: PostgreSQL stored sensor readings persisted from MQTT-originated measurements. Source: Authors.

Table 7.1: Main database groups

Group	Tables / examples	Purpose
Agricultural structure	users, farms, plots, management_zones, soil_profiles, crops	Represent physical organization of the farm and monitored zones
Acquisition	sensors, sensor_readings, weather_records, device_health_logs	Store IoT measurements and device condition
Forecasting	forecast_locations, weather_forecasts, irrigation_forecasts	Support weather-aware planning
Digital Twin	digital_twins, zone_states, zone_state_history	Maintain virtual state and historical evolution
Prediction/simulation	prediction_results, twin_predictions, simulation_scenarios, simulation_results	Store ML and what-if outputs
Decision/control	recommendations, alerts, irrigation_events, control_commands, execution_feedback	Trace decisions and physical action

Chapter 8

Digital Twin and Machine Learning Layer

8.1 8.1 Introduction

This chapter presents the Digital Twin and machine learning layer of Agrio. It explains how sensor readings become a virtual zone state, how Random Forest models predict irrigation need, how the system simulates future moisture evolution, and how recommendations are generated.

8.2 8.2 Virtual zone state

The zone state is the central representation of the Digital Twin. It stores current moisture, soil temperature, water balance, stress level, irrigation need, recommended action, pump status, synchronization information, prediction confidence, and timestamps. This state is updated whenever new sensor data arrives or when prediction/simulation processes run.

8.3 8.3 Dataset adaptation

The dataset adaptation script prepares raw data for model training and inference. It normalizes soil moisture, parses light intensity, converts wind direction, extracts date/time features, adds cyclic features, creates missing-value indicators, and derives helper labels such as irrigation need and favorable irrigation conditions.

Insert Figure 8.2 here: Dataset adaptation and feature engineering workflow used before model training.

Figure 8.1: Dataset adaptation and feature engineering workflow used before model training. Source: Authors.

8.4 8.4 Feature engineering

Features include temporal variables, environmental variables, soil variables, rainfall indicators, wind information, missing-value flags, cyclic hour/month encodings, and wind direction encodings. These

features are suitable for Random Forest because the model can handle nonlinear interactions among tabular variables.

8.5 8.5 Random Forest models

Two Random Forest models are used. The first predicts whether irrigation is needed. The second suggests an irrigation hour. The model metadata reports high evaluation performance for irrigation need and lower but usable performance for suggested hour. These metrics are treated as dataset-level evaluation results, not as field-validated agronomic performance.

Insert Figure 8.1 here: Machine learning pipeline from dataset adaptation to Random Forest inference.

Figure 8.2: Machine learning pipeline from dataset adaptation to Random Forest inference. Source: Authors.

8.6 8.6 ML inference service

The ML service exposes endpoints such as `/health`, `/metadata`, and `/predict`. The backend prepares a feature payload and sends it to the ML service. The response includes irrigation need, confidence, suggested hour, suggested-hour confidence, recommended action, and features used.

Insert Figure 8.3 here: ML service health endpoint showing model loading status.

Figure 8.3: ML service health endpoint showing model loading status. Source: Authors.

Insert Figure 8.4 here: ML prediction response for irrigation need and suggested irrigation hour.

Figure 8.4: ML prediction response for irrigation need and suggested irrigation hour. Source: Authors.

8.7 8.7 Digital Twin state update

After ML inference, the backend stores prediction results and updates the zone state with AI fields. The state history table records previous states, enabling temporal analysis and validation. This creates continuity between measurements, predictions, and dashboard display.

Insert Figure 8.5 here: Digital Twin state update flow from sensor readings to synchronized zone state.

Figure 8.5: Digital Twin state update flow from sensor readings to synchronized zone state. Source: Authors.

8.8 8.8 What-if simulation

The what-if simulation compares future moisture evolution under different scenarios. Inputs include zone, irrigation duration, projection horizon, and optional flow-rate override. Outputs include

simulated moisture, water balance, 24-hour and 48-hour projection, recommended action, and explanation.

Insert Figure 8.6 here: What-if simulation logic comparing no-irrigation, with-irrigation, and ML-selected paths.

Figure 8.6: What-if simulation logic comparing no-irrigation, with-irrigation, and ML-selected paths. Source: Authors.

8.9 8.9 Recommendation generation

The recommendation engine combines Digital Twin state, ML prediction, simulation result, forecast information, and rule-based constraints. Recommendations can indicate keep off, irrigate now, schedule irrigation, or monitor. They include priority, duration, reason, confidence, and status.

8.10 8.10 Explainability

Agrio improves explainability through dashboard text, confidence values, comparison charts, and scenario explanations. This is important because farmers and jury members must understand why the system recommends or rejects irrigation.

8.11 8.11 Limitations

The ML layer is limited by the dataset used, the controlled prototype context, and the absence of long-term multi-crop field validation. Model results must therefore be interpreted cautiously. Future work should include retraining using real field data and comparing Random Forest with gradient boosting, LSTM, and reinforcement learning.

8.12 8.12 Conclusion

This chapter described the intelligent layer of Agrio. The Digital Twin state, Random Forest models, simulation, recommendation engine, and explanation mechanisms together transform raw sensor data into actionable irrigation support.

Table 8.1: Features used by the need_irrigation model

Feature group	Examples	Purpose
Temporal	year, month, day, hour, cyclic hour/month	Capture seasonality and time-of-day effects
Weather	temperature, humidity, rainfall, wind speed, wind direction	Represent atmospheric demand and rain conditions
Soil	soil_moisture_percent, soil_moisture_fraction, soil_temperature	Represent water availability and soil state
Missing flags	soil_temperature_missing, light_missing, pressure_missing	Inform the model about absent measurements
Derived	irrigation history, favorable condition indicators	Encode operational context

Table 8.2: Features used by the suggested_hour model

Feature group	Examples	Purpose
Base features	same temporal, weather, and soil features	Represent current environment
Decision features	need_irrigation, is_favorable	Use the irrigation decision context
Cyclic features	sin/cos hour and month	Preserve circular time behavior
Wind encoding	direction degrees, sin/cos direction	Represent wind direction numerically

Table 8.3: Model performance metrics from project metadata

Model	Main task	Reported metric	Interpretation
Random Forest need_irrigation	Binary irrigation need prediction	High accuracy reported in metadata	Good dataset-level performance; field validation still required
Random Forest suggested_hour	Suggested irrigation hour prediction	Moderate-to-high accuracy reported in metadata	Useful for planning support, should be validated with real schedules
Rule-based fallback	Fallback decision when ML is unavailable	Not an ML metric	Improves system continuity and safety

Insert Figure 8.7a here: Feature importance of the Random Forest need_irrigation model.

Figure 8.7: Feature importance of the Random Forest need_irrigation model. Source: Authors.

Insert Figure 8.7b here: Feature importance of the Random Forest suggested_hour model.

Figure 8.8: Feature importance of the Random Forest suggested_hour model. Source: Authors.

Insert Figure 8.8a here: Confusion matrix of the need_irrigation Random Forest model.

Figure 8.9: Confusion matrix of the need_irrigation Random Forest model. Source: Authors.

Insert Figure 8.8b here: Confusion matrix of the suggested_hour Random Forest model.

Figure 8.10: Confusion matrix of the suggested_hour Random Forest model. Source: Authors.

Chapter 9

User Interfaces and Irrigation Control

9.1 9.1 Introduction

The user interface is the operational face of the Digital Twin. It transforms sensor data, ML outputs, simulation results, recommendations, schedules, and actuator status into understandable information. Agrio includes a Next.js web dashboard and a Flutter mobile application.

9.2 9.2 Frontend architecture

The frontend is organized into pages for dashboard, forecast, Digital Twin simulation, schedules, validated decisions, and zone details. It communicates with the backend through REST APIs using a configured API base URL and periodically refreshes dashboard data.

9.3 9.3 Operations dashboard

The operations dashboard shows MQTT connection, active zone coverage, moisture snapshot, sensor health, twin status, plot name, last update, and quick actions. It gives the user a high-level overview of the system state.

9.4 9.4 Zone monitoring

The zone selection panel allows the user to inspect the Digital Twin state of a specific management zone. The current prototype has one experimental zone. The card shows moisture, health, and last update.

9.5 9.5 AI recommendation preview

The recommendation preview presents the current decision, confidence, suggested time, duration, and explanation. The user can validate the decision or manually start the pump. This supports human-in-the-loop decision-making.

9.6 9.6 Pump and actuator control

The pump control card displays WeMos device ID, pump D5 status, lamp D3 status, last command, and manual control buttons. Commands are sent to the backend, which publishes MQTT commands to the WeMos.

9.7 9.7 Forecast workspace

The forecast page generates weather-aware irrigation planning. It uses weather data and zone context to indicate irrigation days, monitoring days, skipped days, risk level, and recommended durations. This extends decision support beyond the current sensor state.

9.8 9.8 Digital Twin simulation workspace

The simulation page guides the user through selecting a zone, reading live state, setting scenario inputs, running simulation, comparing results, and acting on outputs. It displays a pseudo-3D field view and moisture projection chart.

9.9 9.9 Schedules page

The schedules page allows creation and management of irrigation events. Scheduled events are stored in the database and later executed by the scheduler worker through MQTT commands.

9.10 9.10 Validated decisions page

The validated decisions page records approved recommendations and decision history. It improves traceability by showing what was validated, why, and with what operational parameters.

9.11 9.11 Mobile application

The Flutter mobile application gives access to monitoring and selected decision functions from a phone. It is especially important because irrigation systems are often managed outside the desktop environment.

9.12 9.12 Conclusion

The interface layer makes the Digital Twin usable. It connects monitoring, recommendation, simulation, forecasting, scheduling, validation, and manual control in a single workflow.

Insert Figure 9.1 here: Agrio operations dashboard showing MQTT connection, active zone coverage, moisture snapshot, sensor health, and Digital Twin status.

Figure 9.1: Agrio operations dashboard showing MQTT connection, active zone coverage, moisture snapshot, sensor health, and Digital Twin status. Source: Authors.

Insert Figure 9.2 here: Zone selection panel for inspecting the Digital Twin state, recommendations, commands, and historical data.

Figure 9.2: Zone selection panel for inspecting the Digital Twin state, recommendations, commands, and historical data. Source: Authors.

Insert Figure 9.3 here: Current field status and AI recommendation preview for the monitored irrigation zone.

Figure 9.3: Current field status and AI recommendation preview for the monitored irrigation zone. Source: Authors.

Insert Figure 9.4 here: Pump and actuator control interface with WeMos status and weather station summary.

Figure 9.4: Pump and actuator control interface with WeMos status and weather station summary. Source: Authors.

Insert Figure 9.5 here: Forecast workspace used to generate weather-aware irrigation planning decisions.

Figure 9.5: Forecast workspace used to generate weather-aware irrigation planning decisions. Source: Authors.

Insert Figure 9.6 here: Digital Twin simulation workspace for running what-if irrigation scenarios.

Figure 9.6: Digital Twin simulation workspace for running what-if irrigation scenarios. Source: Authors.

Insert Figure 9.7 here: Moisture projection chart comparing no-irrigation, with-irrigation, and ML-selected trajectories.

Figure 9.7: Moisture projection chart comparing no-irrigation, with-irrigation, and ML-selected trajectories. Source: Authors.

Insert Figure 9.8 here: Irrigation schedules page for planning and managing pump execution events.

Figure 9.8: Irrigation schedules page for planning and managing pump execution events. Source: Authors.

Insert Figure 9.9 here: Validated decisions interface for tracking approved irrigation recommendations.

Figure 9.9: Validated decisions interface for tracking approved irrigation recommendations. Source: Authors.

Insert Figure 9.10 here: Mobile application screens providing access to monitoring and irrigation decision functions.

Figure 9.10: Mobile application screens providing access to monitoring and irrigation decision functions. Source: Authors.

Part V

Deployment, Validation, and Discussion

Chapter 10

Deployment Environment

10.1 10.1 Introduction

Deployment is a critical engineering part of the project. Agrio is composed of several services that must run together: database, backend, ML service, frontend, scheduler, MQTT configuration, and mobile access support. Docker Compose is used to coordinate these services.

10.2 10.2 Docker Compose stack

The Docker Compose stack includes PostgreSQL, the ML service, FastAPI backend, frontend, irrigation scheduler, ngrok, and optional worker profiles. Each service has a clear responsibility and communicates through internal network links and environment variables.

10.3 10.3 PostgreSQL service

The database service initializes the `smart_irrigation` schema and persists structured data. It is the backbone of traceability because it records physical measurements, virtual states, predictions, simulations, recommendations, commands, and feedback.

10.4 10.4 ML service

The ML service loads trained model files and exposes health, metadata, and prediction endpoints. This separated deployment makes it possible to update or retrain models independently from the backend.

10.5 10.5 Backend service

The backend service exposes the API, subscribes to MQTT when configured, calls the ML service, interacts with PostgreSQL, and provides data to the frontend and mobile application.

10.6 10.6 Frontend service

The frontend is exposed on port 3000 and communicates with the backend. It provides the web dashboard, forecast, simulation, schedules, and validated decisions.

10.7 10.7 Scheduler worker

The irrigation scheduler runs independently from HTTP request handling. It checks due irrigation events, sends pump ON/OFF commands, updates event status, and records feedback.

10.8 10.8 Ngrok and mobile access

Ngrok can expose local services for mobile testing. This is useful when the Flutter application must access the backend from a mobile device during demonstrations.

10.9 10.9 Deployment limitations

The deployment is suitable for prototype demonstration but not yet for production. A production environment would require HTTPS, secure credentials, role-based access control, backups, monitoring, and hardened MQTT authentication.

10.10 10.10 Conclusion

The deployment chapter shows that Agrio is implemented as a reproducible multi-service platform rather than a single script or static dashboard.

Insert Figure 10.1 here: Docker Compose service organization of the deployed Agrio platform.

Figure 10.1: Docker Compose service organization of the deployed Agrio platform. Source: Authors.

Insert Figure 10.2 here: Running Docker containers of the deployed Digital Twin smart irrigation platform.

Figure 10.2: Running Docker containers of the deployed Digital Twin smart irrigation platform. Source: Authors.

Insert Figure 10.3 here: Deployment communication flow between frontend, backend, database, ML service, MQTT broker, and external APIs.

Figure 10.3: Deployment communication flow between frontend, backend, database, ML service, MQTT broker, and external APIs. Source: Authors.

Insert Figure 10.4 here: Ngrok tunnel used to expose local services for mobile access during prototype testing.

Figure 10.4: Ngrok tunnel used to expose local services for mobile access during prototype testing. Source: Authors.

Table 10.1: Deployment services and roles

Service	Role	Evidence to include
postgres	Stores smart_irrigation schema	Docker container and database screenshot
ml-service	Loads Random Forest models and predicts irrigation outputs	/health, /metadata, /predict screenshots
backend	API, MQTT ingestion, twin update, recommendations, commands	FastAPI /docs and logs
frontend	Web dashboard and user workspaces	Dashboard screenshots
irrigation-scheduler	Executes scheduled pump events	Scheduler logs or event status
ngrok	Mobile access tunnel	Ngrok forwarding screenshot
mqtt-worker/simulator	Optional profiles for ingestion/simulation	Docker profile or logs

Chapter 11

Results, Validation, and Discussion

11.1 11.1 Introduction

Validation in this thesis focuses on technical correctness of the prototype workflow. The objective is to demonstrate that data can move from physical sensors to MQTT, database, Digital Twin state, ML prediction, dashboard, simulation, recommendation, and pump command.

11.2 11.2 Prototype validation context

The system was tested on a controlled experimental prototype, not on a commercial farm. Therefore, validation proves functional integration and workflow feasibility. Agronomic water-saving claims require future field experiments.

11.3 11.3 Sensor data acquisition

The HiveMQ screenshot shows sensor data published on `sensors/data` and WeMos status published on `smart_irrigation/wemos1/status`. This proves that the communication layer receives physical and actuator information.

11.4 11.4 Database persistence

The database screenshot shows stored sensor readings with `id`, `zone_id`, `sensor_type`, `value`, `quality_flag`, `source`, and `timestamp`. This proves that MQTT-originated measurements are persisted in PostgreSQL.

11.5 11.5 Dashboard synchronization

The dashboard shows the latest moisture, zone state, sensor health, twin status, environment snapshot, and recommendation. This proves that backend data is exposed to the user interface.

11.6 11.6 ML prediction validation

The ML service should be validated through /health, /metadata, and /predict screenshots. These show that models are loaded, metadata is available, and prediction output contains irrigation need, suggested hour, confidence, and recommended action.

11.7 11.7 Simulation validation

The Digital Twin simulation result compares no-irrigation, with-irrigation, and ML-selected paths. This proves that the platform can run what-if analysis rather than only display current values.

11.8 11.8 Pump control validation

Pump control is validated through MQTT command evidence and WeMos status feedback. The key expected sequence is ON_D5 command, D5 ON status, OFF_D5 command, and D5 OFF status.

11.9 11.9 Discussion of results

The results demonstrate end-to-end integration. The project succeeds in implementing the core Digital Twin loop in a controlled environment. The main limitation is that the loop has not yet been validated across seasons, crops, and real field variability.

11.10 11.10 Comparison with literature

Compared with MQTT smart irrigation systems, Agrio adds Digital Twin state and simulation. Compared with ML-only irrigation systems, Agrio integrates prediction into a deployed backend and dashboard. Compared with field-validated DT systems, Agrio is less agronomically validated but more transparent as a student engineering prototype.

11.11 11.11 Limitations

Limitations include controlled prototype scale, limited long-term data, dataset adaptation constraints, lack of satellite/UAV integration, limited security hardening, and the need for sensor calibration. These are normal for a final-year prototype but must be addressed before real deployment.

11.12 11.12 Future work

Future work includes field deployment, multiple zones, crop-specific calibration, satellite/UAV data assimilation, improved evapotranspiration modeling, model retraining, reinforcement learning optimization, emergency stop mechanisms, cybersecurity, and farmer usability studies.

11.13 11.13 Conclusion

This chapter validated the main technical workflow and discussed limitations. The system demonstrates a strong proof of concept for a Digital Twin-based intelligent irrigation platform.

Table 11.1: Test scenario summary

Test ID	Objective	Expected result	Evidence figure	Status
T1	MQTT sensor payload reception	sensors/data message received	Figure 11.1	To capture/insert
T2	Database persistence	sensor_readings rows created	Figure 11.2	Captured/insert
T3	Dashboard live update	dashboard reflects latest state	Figure 11.3	Captured/insert
T4	ML prediction request	predict response returns decision and confidence	Figure 11.4	To capture/insert
T5	Digital Twin synchronization	zone_state updated	Figure 11.3 / 11.5	Captured/insert
T6	What-if simulation	scenario output and chart generated	Figure 11.5	Captured/insert
T7	Recommendation generation	recommendation pending/validated	Figure 9.3 / 9.9	To capture/insert
T8	Manual pump ON command	ON_D5 published and D5 ON status	Figures 11.6-11.7	To capture/insert
T9	Manual pump OFF command	OFF_D5 published and D5 OFF status	Figure 11.8	To capture/insert
T10	Scheduled irrigation event	scheduler records execution feedback	Figure 11.9	To capture/insert
T11	Forecast generation	forecast decision visible	Figure 9.5	To capture/insert
T12	API documentation	FastAPI /docs accessible	Figure 7.3	Captured/insert

Insert Figure 11.1 here: Sensor MQTT message received on the sensors/data topic.

Figure 11.1: Sensor MQTT message received on the sensors/data topic. Source: Authors.

Insert Figure 11.2 here: Database persistence proof showing stored sensor readings from MQTT-originated data.

Figure 11.2: Database persistence proof showing stored sensor readings from MQTT-originated data. Source: Authors.

Insert Figure 11.3 here: Dashboard update reflecting the latest stored sensor and Digital Twin data.

Figure 11.3: Dashboard update reflecting the latest stored sensor and Digital Twin data. Source: Authors.

Insert Figure 11.4 here: ML prediction result showing irrigation need, suggested hour, confidence, and action.

Figure 11.4: ML prediction result showing irrigation need, suggested hour, confidence, and action. Source: Authors.

Insert Figure 11.5 here: Digital Twin simulation result comparing projected moisture and recommended action.

Figure 11.5: Digital Twin simulation result comparing projected moisture and recommended action. Source: Authors.

Insert Figure 11.6 here: Pump ON command sent to the WeMos actuator through MQTT.

Figure 11.6: Pump ON command sent to the WeMos actuator through MQTT. Source: Authors.

Insert Figure 11.7 here: WeMos status feedback after pump activation.

Figure 11.7: WeMos status feedback after pump activation. Source: Authors.

Insert Figure 11.8 here: Pump OFF command sent to the WeMos actuator through MQTT.

Figure 11.8: Pump OFF command sent to the WeMos actuator through MQTT. Source: Authors.

Insert Figure 11.9 here: Execution feedback or irrigation event status recorded after pump command execution.

Figure 11.9: Execution feedback or irrigation event status recorded after pump command execution. Source: Authors.

General Conclusion

This thesis presented the design and deployment of Agrio, a Digital Twin-based intelligent irrigation platform for precision agriculture. The work was developed as a combined Master 2 and Engineering degree thesis, bringing together scientific analysis and practical implementation. The scientific part studied Digital Twins, IoT-based smart irrigation, machine learning irrigation decisions, and agricultural water management. The engineering part implemented an end-to-end prototype that connects physical sensing, radio-frequency communication, MQTT messaging, backend processing, database persistence, ML inference, Digital Twin simulation, user interfaces, scheduling, and pump control.

The state-of-the-art review showed that Digital Twins are increasingly studied in agriculture but remain difficult to deploy due to data heterogeneity, biological complexity, cost, connectivity, calibration, and limited long-term validation. The literature also shows that many systems focus on monitoring or decision support but do not always implement a complete sensing-to-actuation loop. In this context, Agrio contributes an integrated architecture and prototype demonstrating how a human-supervised Digital Twin irrigation system can be built.

The implemented system uses a Libelium Smart Agriculture kit, XBee communication, Raspberry Pi gateway, HiveMQ/MQTT, FastAPI backend, PostgreSQL database, Random Forest-based ML service, Digital Twin state and simulation services, Next.js dashboard, Flutter mobile application, WeMos actuator, and water pump. Sensor data are parsed, normalized, stored, and converted into a synchronized virtual zone state. The ML layer predicts irrigation need and suggested timing. The simulation layer compares future moisture evolution under different scenarios. The recommendation layer supports human validation. The scheduling and device-control layer can send commands to the pump.

The main result is a working proof of concept that goes beyond classical IoT monitoring. The system provides synchronization, prediction, simulation, recommendation, validation, scheduling, and actuator feedback. It therefore demonstrates a practical implementation of Digital Twin principles in a smart irrigation context.

Nevertheless, the work remains a prototype. It was validated in a controlled experimental environment, not over multiple seasons in a commercial farm. Its ML models require further field validation and retraining. Security, authentication, sensor calibration, agronomic modeling, remote sensing integration, and multi-zone optimization require future work. These limitations are important and should be considered as the next steps toward a production-grade agricultural Digital Twin.

Future improvements include deploying the system in a real field, collecting long-term data, calibrating soil and crop parameters, integrating Open-Meteo forecasts more deeply, adding satellite or UAV imagery, improving the water balance model, comparing Random Forest with advanced models, implementing reinforcement learning for irrigation optimization, strengthening cybersecurity, adding an emergency stop mechanism, and evaluating usability with farmers and agronomists.

Bibliography

- [1] A. Subeesh and N. Chauhan, “Agricultural digital twin for smart farming: A review,” *Green Technologies and Sustainability*, 2026.
- [2] R. Zhang, H. Zhu, Q. Chang, and Q. Mao, “A comprehensive review of digital twins technology in agriculture,” *Agriculture*, 2025.
- [3] R. Gund, C. M. Badgular, S. Samiappan, and S. Jagadamma, “Application of digital twin technology in smart agriculture: A bibliometric review,” *Agriculture*, 2025.
- [4] M. Awais, X. Wang, S. Hussain, F. Aziz, and M. Q. Mahmood, “Advancing precision agriculture through digital twins and smart farming technologies: A review,” *AgriEngineering*, 2025.
- [5] M. Tsaousidis, T. Kalampokas, E. Vrochidou, and G. A. Papakostas, “Ai-enabled digital twins in agriculture,” *AI*, 2026.
- [6] R. Ahsen *et al.*, “Harnessing digital twins for sustainable agricultural water management: A systematic review,” *Applied Sciences*, 2025.
- [7] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, “Iot-based smart irrigation systems: An overview on the recent trends on sensors and iot systems for irrigation in precision agriculture,” *Sensors*, 2020.
- [8] K. Obaideen *et al.*, “An overview of smart irrigation systems using iot,” *Energy Nexus*, 2022.
- [9] M. Del-Coco, M. Leo, and P. Carcagnì, “Machine learning for smart irrigation in agriculture: How far along are we?” *Information*, 2024.
- [10] A. Morchid, Z. Said, and H. Tairi, “Innovative applications of internet of things and machine learning in sustainable agricultural irrigation management: Benefits and challenges,” *Smart Agricultural Technology*, 2026.
- [11] R. K. Kodali and B. S. Sarjerao, “A low cost smart irrigation system using mqtt protocol,” in *IEEE Region 10 Symposium*, 2017.
- [12] C. Balaceanu, G. Suciu, and I. Marcu, “Libelium-based iot monitoring solution for precision agriculture,” *Journal of E-Technology*, 2019.
- [13] S. Millán, C. Montesinos, J. Casadesús, J. M. Vadillo, and C. Campillo, “Use of a digital twin for water efficient management in a processing tomato commercial farm,” *Agronomy*, 2025.
- [14] J. Bellvert *et al.*, “Assimilation of sentinel-2 biophysical variables into a digital twin for the automated irrigation scheduling of a vineyard,” *Water*, 2023.

- [15] H. Manocha, S. K. Sood, and M. Bhatia, “Iot-digital twin-inspired smart irrigation approach for optimal water utilization,” *Sustainable Computing: Informatics and Systems*, 2024.
- [16] F. Tancredi, P. Preite, and G. Vignali, “Digital twin enhanced with machine learning algorithms for irrigation management using sensor data,” in *Procedia Computer Science*, 2025.
- [17] Qin *et al.*, “Digital twin-enabled intelligent irrigation-drainage system for precision water-salt management in saline agroecosystems,” *Agricultural Water Management*, 2025.
- [18] Liu *et al.*, “Digital twin technology for cotton canopy development: A water-stress perspective,” *Smart Agricultural Technology*, 2025.
- [19] E. Hachimi *et al.*, “Towards collective intelligence in agriculture: Deep reinforcement learning and digital twins for efficient management of collective irrigation water distribution systems,” *Smart Agricultural Technology*, 2025.
- [20] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*. Wiley, 2021.
- [21] L. Breiman, “Random forests,” *Machine Learning*, 2001.
- [22] FastAPI Documentation, “Fastapi framework and automatic openapi documentation,” 2026.
- [23] Docker Documentation, “Docker compose overview,” 2026.
- [24] HiveMQ, “Mqtt essentials and publish/subscribe messaging,” 2026.
- [25] Open-Meteo Documentation, “Weather forecast api,” 2026.
- [26] Libelium, “Smart agriculture iot vertical kit and plug & sense sensing platform,” 2017.

Appendix A

Figure Placement Checklist

Figure 0.1 - Global view of the thesis methodology from literature review to prototype validation. Source: Authors. Placement: Introduction. [Insert final figure here]

Figure 1.1 - Conceptual scope of precision agriculture and smart irrigation. Source: Authors. Placement: Chapter 1. [Insert final figure here]

Figure 1.2 - Generic communication chain for IoT-based smart irrigation. Source: Authors. Placement: Chapter 1. [Insert final figure here]

Figure 1.3 - Pump control concept using WeMos, relay, and MQTT command/status topics. Source: Authors. Placement: Chapter 1. [Insert final figure here]

Figure 2.1 - Digital Model, Digital Shadow, and Digital Twin maturity levels. Source: Authors. Placement: Chapter 2. [Insert final figure here]

Figure 2.2 - Layered architecture of an agricultural Digital Twin. Source: Authors. Placement: Chapter 2. [Insert final figure here]

Figure 3.1 - Literature review methodology and thematic classification. Source: Authors. Placement: Chapter 3. [Insert final figure here]

Figure 3.2 - Example of agricultural Digital Twin overview from literature [insert or redraw based on Subeesh and Chauhan, Fig. 1]. Source: Adapted/redrawn from [1]. Placement: Chapter 3. [Insert final figure here]

Figure 3.3 - Generic smart irrigation system architecture from ML literature [insert/redraw based on Del-Coco et al., Fig. 1]. Source: Adapted/redrawn from [9]. Placement: Chapter 3. [Insert final figure here]

Figure 4.1 - Positioning of Agrio between Digital Model, Digital Shadow, and operational Digital Twin. Source: Authors. Placement: Chapter 4. [Insert final figure here]

Figure 5.1 - Use-case diagram of the proposed Digital Twin smart irrigation system. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 5.2 - Activity diagram of sensor ingestion, Digital Twin update, AI decision-making, and pump control. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 5.3 - Agricultural domain class diagram representing farms, plots, zones, soil profiles, crops, and irrigation networks. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 5.4 - Sensor, weather, and forecasting data model used by the smart irrigation platform. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 5.5 - Digital Twin, prediction, and simulation data model. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 5.6 - Decision, recommendation, scheduling, command, and feedback data model. Source: Authors. Placement: Chapter 5. [Insert final figure here]

Figure 6.1 - Global overview of the Agrio Digital Twin smart irrigation architecture. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.2 - Physical and communication architecture of the controlled smart irrigation prototype. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.3 - MQTT topic architecture for sensor ingestion, actuator commands, and WeMos status feedback. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.4 - Internal backend architecture of the FastAPI-based Digital Twin platform. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.5 - Digital Twin processing architecture from live sensor data to simulation and recommendation. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.6 - Docker Compose deployment architecture of the Agrio platform. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 6.7 - Detailed technical architecture of the implemented Digital Twin smart irrigation platform. Source: Authors. Placement: Chapter 6. [Insert final figure here]

Figure 7.6 - Example MQTT payloads used for sensor ingestion, WeMos status feedback, and actuator commands. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.5a - ERD of the agricultural structure tables. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.5b - ERD of the sensor, weather, and forecasting tables. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.5c - ERD of the Digital Twin, prediction, and simulation tables. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.5d - ERD of the recommendation, scheduling, command, and feedback tables. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.1 - Runtime sequence of MQTT data ingestion, ML prediction, dashboard polling, and WeMos actuator command. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.2 - Live MQTT payloads received through HiveMQ for sensor data and WeMos status. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.3 - FastAPI OpenAPI documentation exposing the REST endpoints of the backend. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 7.4 - PostgreSQL stored sensor readings persisted from MQTT-originated measurements. Source: Authors. Placement: Chapter 7. [Insert final figure here]

Figure 8.2 - Dataset adaptation and feature engineering workflow used before model training. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.1 - Machine learning pipeline from dataset adaptation to Random Forest inference. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.3 - ML service health endpoint showing model loading status. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.4 - ML prediction response for irrigation need and suggested irrigation hour. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.5 - Digital Twin state update flow from sensor readings to synchronized zone state. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.6 - What-if simulation logic comparing no-irrigation, with-irrigation, and ML-selected paths. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.7a - Feature importance of the Random Forest need_irrigation model. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.7b - Feature importance of the Random Forest suggested_hour model. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.8a - Confusion matrix of the need_irrigation Random Forest model. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 8.8b - Confusion matrix of the suggested_hour Random Forest model. Source: Authors. Placement: Chapter 8. [Insert final figure here]

Figure 9.1 - Agrio operations dashboard showing MQTT connection, active zone coverage, moisture snapshot, sensor health, and Digital Twin status. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.2 - Zone selection panel for inspecting the Digital Twin state, recommendations, commands, and historical data. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.3 - Current field status and AI recommendation preview for the monitored irrigation zone. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.4 - Pump and actuator control interface with WeMos status and weather station summary. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.5 - Forecast workspace used to generate weather-aware irrigation planning decisions. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.6 - Digital Twin simulation workspace for running what-if irrigation scenarios. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.7 - Moisture projection chart comparing no-irrigation, with-irrigation, and ML-selected trajectories. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.8 - Irrigation schedules page for planning and managing pump execution events. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.9 - Validated decisions interface for tracking approved irrigation recommendations. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 9.10 - Mobile application screens providing access to monitoring and irrigation decision functions. Source: Authors. Placement: Chapter 9. [Insert final figure here]

Figure 10.1 - Docker Compose service organization of the deployed Agrio platform. Source: Authors. Placement: Chapter 10. [Insert final figure here]

Figure 10.2 - Running Docker containers of the deployed Digital Twin smart irrigation platform. Source: Authors. Placement: Chapter 10. [Insert final figure here]

Figure 10.3 - Deployment communication flow between frontend, backend, database, ML service, MQTT broker, and external APIs. Source: Authors. Placement: Chapter 10. [Insert final figure here]

Figure 10.4 - Ngrok tunnel used to expose local services for mobile access during prototype testing. Source: Authors. Placement: Chapter 10. [Insert final figure here]

Figure 11.1 - Sensor MQTT message received on the sensors/data topic. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.2 - Database persistence proof showing stored sensor readings from MQTT-originated data. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.3 - Dashboard update reflecting the latest stored sensor and Digital Twin data. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.4 - ML prediction result showing irrigation need, suggested hour, confidence, and action. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.5 - Digital Twin simulation result comparing projected moisture and recommended action. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.6 - Pump ON command sent to the WeMos actuator through MQTT. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.7 - WeMos status feedback after pump activation. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.8 - Pump OFF command sent to the WeMos actuator through MQTT. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Figure 11.9 - Execution feedback or irrigation event status recorded after pump command execution. Source: Authors. Placement: Chapter 11. [Insert final figure here]

Appendix B

Full Class Diagram

Insert Figure A.1 here: Complete class diagram of the implemented Digital Twin smart irrigation platform.

Figure B.1: Complete class diagram of the implemented Digital Twin smart irrigation platform.
Source: Authors.

Appendix C

Full Detailed Architecture

Insert Figure A.2 here: Large detailed architecture of the implemented Agrio platform.

Figure C.1: Large detailed architecture of the implemented Agrio platform. Source: Authors.

Appendix D

Complete Database ERD

Insert Figure A.3 here: Complete database ERD of the smart_irrigation schema.

Figure D.1: Complete database ERD of the smart_irrigation schema. Source: Authors.

Appendix E

Example MQTT Payloads

Sensor data payload example:

```
{"device_id":"4D6F10FDC337DEA1","humidity":44.8,"leaf_wetness":0.0,"light_intensity":"4.67T",
  "pressure":0.96,"rainfall_current":0.0,"rainfall_day":0.0,"rainfall_hour":0.0,"
  soil_moisture":0.27,"soil_temperature":22.77,"temperature":24.08,"timestamp
  ":"2026-04-22T14:17:12.143199Z","wind_direction":"SE","wind_speed":0.0}
```

WeMos status payload example:

```
{"device_id":"wemos1","D5":"OFF","D3":"OFF"}
```

Command examples: ON_D5, OFF_D5, ON_D3, OFF_D3, ALL_ON, ALL_OFF.

Appendix F

Missing Visual Capture Guide

- Open the forecast page and capture the full weather-aware workspace.
- Open the schedules page and capture the event list and create/edit form.
- Open the validated decisions page and capture validated recommendations.
- Capture the Flutter mobile dashboard and zone/twin state screens.
- Run docker ps or open Docker Desktop and capture running containers.
- Open the ML service /health endpoint and capture the JSON response.
- Open the ML service /metadata endpoint and capture model features and metrics.
- Call /predict using Swagger or Postman and capture the prediction output.
- Send ON_D5 and capture MQTT command plus WeMos D5 ON status.
- Send OFF_D5 and capture MQTT command plus WeMos D5 OFF status.

Appendix G

Thesis Expansion Notes for Final Editing

When final figures are available, each placeholder should be replaced by a high-resolution image. Captions should remain under the figure, and the figure must be cited in the surrounding paragraph before it appears. For figures adapted from research papers, redraw the figure in the thesis style and cite the source in the caption. For screenshots and prototype photos produced by the students, use Source: Authors.

The final defense version may simplify the thesis architecture diagrams. The full detailed architecture is useful for the manuscript and appendix, but defense slides should use the split diagrams: global overview, physical communication, MQTT topics, backend internal architecture, Digital Twin processing, and deployment architecture.

The final experimental chapter should be updated with actual observations once the missing screenshots are captured. Each test should include input, expected result, observed result, evidence figure, and status. If a test was not performed, it must be marked as future validation rather than presented as completed.

Appendix H

Detailed Paper-by-Paper Literature Matrix

This appendix provides an expanded paper-by-paper analysis used to support Chapters 3 and 4. It is included so that the main body remains readable while the scientific basis of the positioning remains explicit. Each study is analyzed according to its objective, architecture, data sources, intelligence layer, Digital Twin maturity, relevance to Agrio, and limitation.

H.1 Subeesh and Chauhan, 2026 [1]

Focus: Agricultural Digital Twin review

Main contribution: The paper reviews Agricultural Digital Twins as virtual representations of farms and processes supported by IoT, AI, AR/XR, edge computing, and related technologies. It emphasizes that DTs enable real-time monitoring, simulation, optimization, and decision support, but also that agriculture remains an emerging and complex DT domain.

Use in this thesis: This source supports the broad theoretical framing of Agrio. It justifies presenting the system as an agricultural DT prototype and highlights why IoT and AI are necessary enablers.

Relation to Agrio: Agrio aligns with this review by combining physical sensing, virtual states, AI prediction, simulation, and dashboard decision support. However, Agrio remains narrower because it targets irrigation only.

H.2 Zhang et al., 2025 [2]

Focus: Comprehensive DT review in agriculture

Main contribution: The study clarifies Digital Model, Digital Shadow, and Digital Twin as maturity levels. It also presents perception, network, and application layers. It emphasizes IoT, cloud/edge computing, data acquisition, simulation, modeling, and agricultural decision support.

Use in this thesis: This source is central for defining the maturity of Agrio. The thesis uses it to distinguish monitoring dashboards from operational Digital Twins.

Relation to Agrio: Agrio is placed between a Digital Shadow and an operational human-supervised Digital Twin because it has sensor synchronization and pump-control capability but not

full autonomous production validation.

H.3 Gund et al., 2025 [3]

Focus: Bibliometric review of smart agricultural DTs

Main contribution: The paper maps the growth of Digital Twin publications in agriculture and identifies clusters around precision farming, IoT, AI, cyber-physical systems, controlled environments, sustainability, and food systems.

Use in this thesis: This source supports the claim that Agrio belongs to an active research trend and that Digital Twins in agriculture are becoming system-level rather than task-specific.

Relation to Agrio: Agrio is a practical implementation inside the IoT-AI-precision farming cluster. Its limitation is that it is an academic prototype rather than a large bibliometric trend outcome.

H.4 Awais et al., 2025 [4]

Focus: Precision agriculture through DTs and smart farming

Main contribution: This review studies 167 works from 2018-2025 and stresses the need for an integrated framework connecting IoT, UAVs, ML, remote sensing, and DT ecosystems. It notes that many works remain isolated around specific applications.

Use in this thesis: This source directly supports the integrative ambition of Agrio: instead of a single ML model or dashboard, the project connects multiple layers in one prototype.

Relation to Agrio: Agrio does not yet include UAV or satellite remote sensing, but it provides a clear foundation to integrate them later.

H.5 Tsaousidis et al., 2026 [5]

Focus: AI-enabled DTs in agriculture

Main contribution: The paper states that many agricultural works use the term Digital Twin loosely and sometimes refer to decision support tools rather than synchronized virtual-physical systems. It highlights data integration as a major bottleneck.

Use in this thesis: This source is used to make the thesis scientifically careful. Agrio must demonstrate exactly why it is a DT: live sensor input, virtual state, prediction, simulation, recommendation, and actuation feedback.

Relation to Agrio: Agrio addresses local sensor and ML integration but not yet multi-source satellite/UAV integration or long-term economic evaluation.

H.6 Ahsen et al., 2025 [6]

Focus: Digital Twins for sustainable agricultural water management

Main contribution: This systematic review focuses on water management. It distinguishes monitoring DTs and predictive DTs and highlights applications in irrigation, hydroponics, aquaponics, and vertical farming. It identifies scalability, cost, and data integration barriers.

Use in this thesis: This source directly supports the irrigation-water focus of Agrio and the claim that predictive DTs can support proactive irrigation decisions.

Relation to Agrio: Agrio includes both monitoring and predictive/simulation elements, but field-scale water savings remain future validation.

H.7 García et al., 2020 [7]

Focus: IoT-based smart irrigation systems overview

Main contribution: The review analyzes sensors, actuators, nodes, wireless technologies, monitored variables, and IoT systems used for irrigation in precision agriculture. It highlights cost and implementation challenges.

Use in this thesis: This paper justifies the hardware and communication chapter: smart irrigation needs soil, weather, water, communication, and actuation components.

Relation to Agrio: Agrio uses a more professional sensing kit and MQTT backend pipeline, whereas many reviewed systems are low-cost prototypes with simple thresholds.

H.8 Obaideen et al., 2022 [8]

Focus: Overview of smart irrigation using IoT

Main contribution: The paper links smart irrigation to water-use efficiency and sustainable development. It describes data acquisition, irrigation control, wireless communication, processing, and fault detection as core components.

Use in this thesis: This work supports Chapter 1 and the sustainability motivation of the thesis.

Relation to Agrio: Agrio implements the listed components and adds Digital Twin state, ML prediction, scenario simulation, and human validation.

H.9 Del-Coco et al., 2024 [9]

Focus: Machine learning for smart irrigation

Main contribution: The review defines a smart irrigation architecture with physical, processing, dataset/data-source, and decision layers. It discusses ML algorithms and highlights dataset and long-term data limitations.

Use in this thesis: This is a key source for the ML chapter. It supports the Random Forest choice and the need to present metrics cautiously.

Relation to Agrio: Agrio follows this architecture but adds concrete deployment and Digital Twin synchronization. It still shares the dataset limitation identified by the review.

H.10 Morchid et al., 2026 [10]

Focus: IoT and ML for sustainable irrigation

Main contribution: This systematic review analyzes IoT and ML in irrigation and stresses real-time environmental data, ML decisions, cloud platforms, water conservation, and adoption barriers.

Use in this thesis: This paper supports the project’s use of IoT + ML and the discussion of implementation complexity.

Relation to Agrio: Agrio integrates IoT and ML with a Digital Twin workflow; however, farmer adoption and field economics remain untested.

H.11 Kodali and Sarjerao, 2017 [11]

Focus: Low-cost MQTT smart irrigation

Main contribution: The work uses soil moisture sensing, ESP8266/NodeMCU, MQTT, pump control, and web/mobile visualization. It is simple but highly relevant to actuator control through MQTT.

Use in this thesis: This is a direct implementation reference for the MQTT/WeMos part of Agrio.

Relation to Agrio: Agrio extends this concept by adding gateway processing, PostgreSQL, ML service, Digital Twin simulation, validated recommendations, and scheduling.

H.12 Balaceanu et al., 2019 [12]

Focus: Libelium-based precision agriculture monitoring

Main contribution: The paper uses Libelium to monitor temperature, soil humidity, relative humidity, atmospheric pressure, and solar radiation. It shows that Libelium-based platforms can support precision agriculture monitoring.

Use in this thesis: This supports the choice of Libelium Smart Agriculture hardware.

Relation to Agrio: Agrio uses Libelium not only for monitoring but as the physical data source of a Digital Twin irrigation platform.

H.13 Millán et al., 2025 [13]

Focus: Processing tomato commercial farm Digital Twin

Main contribution: The paper evaluates Irri_DesK in a 15 ha commercial processing tomato plot over two seasons. It integrates soil moisture sensors, climate data, simulation models, and daily irrigation scheduling.

Use in this thesis: This is one of the strongest field-validation references. It shows what a mature irrigation DT can achieve in commercial conditions.

Relation to Agrio: Agrio should be compared honestly: Agrio is more of an end-to-end software/hardware prototype, while this study has stronger agronomic validation.

H.14 Bellvert et al., 2023 [14]

Focus: Sentinel-2 assimilation into vineyard DT

Main contribution: The paper assimilates Sentinel-2 fAPAR into a Digital Twin for automated vineyard irrigation scheduling. It combines remote sensing, soil water balance, sensors, and automated irrigation prescriptions.

Use in this thesis: This supports the future work direction of integrating satellite imagery into Agrio.

Relation to Agrio: Agrio currently lacks remote sensing; adding Sentinel-2 or UAV data would strengthen the Digital Twin maturity.

H.15 Manocha et al., 2024 [15]

Focus: IoT-DT smart irrigation

Main contribution: The paper links sensors and actuators to virtual counterparts, estimates irrigation requirements, and simulates system behavior. It is one of the closest conceptual references to Agrio.

Use in this thesis: This work is a direct comparison point for the proposed architecture.

Relation to Agrio: Agrio's distinct value is the implemented stack: MQTT topics, FastAPI, PostgreSQL, ML service, dashboard, scheduling, and WeMos control.

H.16 Tancredi et al., 2025 [16]

Focus: Digital Twin enhanced with ML for irrigation

Main contribution: The work uses sensor data and machine learning to classify irrigation ON/OFF decisions and evaluates the model through metrics such as accuracy, precision, recall, F1, and confusion matrix.

Use in this thesis: This is a direct comparison for the ML decision layer.

Relation to Agrio: Agrio should include confusion matrices and feature importance figures to reach the same level of ML reporting.

H.17 Qin et al., 2025 [17]

Focus: DT irrigation-drainage for water-salt management

Main contribution: The study addresses irrigation-drainage and water-salt dynamics using IoT, prediction, and DT-assisted management. It is more agronomically specialized than Agrio.

Use in this thesis: This helps discuss advanced irrigation DTs beyond simple moisture thresholds.

Relation to Agrio: Agrio does not handle salinity or drainage; these are future work for water-stressed or saline environments.

H.18 Liu et al., 2025 [18]

Focus: Cotton canopy DT under water stress

Main contribution: This work focuses on plant/canopy modeling under water stress and provides an example of crop-specific DT modeling.

Use in this thesis: This supports the future direction of crop growth and stress modeling.

Relation to Agrio: Agrio currently models zone moisture and decision state rather than detailed crop physiology.

H.19 El Hachimi et al., 2025 [19]

Focus: Digital Twins and reinforcement learning for irrigation distribution

Main contribution: The paper uses DRL and Digital Twins to optimize collective irrigation water distribution. It represents an advanced control direction.

Use in this thesis: This is useful for the future work chapter.

Relation to Agrio: Agrio currently uses Random Forest and rule-based/human-supervised decisions; reinforcement learning would be a future optimization layer.

Appendix I

Detailed Database Table Descriptions

This appendix describes the role of the main PostgreSQL tables used by the Agrio platform. The final ERD should be inserted in Chapter 7 and the complete ERD in Appendix A. The descriptions below are useful for defending the database design during the engineering discussion.

Table I.1: Detailed description of database tables

Table	Role in the platform
users	Stores platform users or operators. It supports ownership and traceability of farms, decisions, and administrative actions.
farms	Represents farm-level entities. A farm is the top agricultural unit and can contain several plots.
plots	Represents physical plots within a farm. In the current prototype, the main plot corresponds to the controlled experimental environment.
management_zones	Represents the basic irrigation decision unit. A zone has its own sensor readings, Digital Twin state, predictions, and recommendations.
soil_profiles	Stores soil properties used to interpret moisture and water balance. Future versions can include texture, field capacity, wilting point, and hydraulic parameters.
crops	Stores crop information used for future crop-specific irrigation decisions.
irrigation_networks	Represents irrigation equipment and flow configuration. It can be extended for valves, pumps, sectors, and pipe characteristics.

Table	Role in the platform
sensors	Stores sensor metadata, type, associated zone, and source device.
sensor_readings	Stores timestamped sensor values. This table is the main persistence target for MQTT-originated measurements.
weather_records	Stores environmental records such as air temperature, humidity, rainfall, wind, pressure, and light intensity.
device_health_logs	Stores sensor/device health status and can support freshness and reliability checks.
forecast_locations	Stores location information used for weather forecast requests.
weather_forecasts	Stores forecast data retrieved from external APIs.
irrigation_forecasts	Stores weather-aware irrigation decisions generated from forecast data.
digital_twins	Represents virtual twins associated with plots or systems.
zone_states	Stores the current synchronized virtual state of each management zone.
zone_state_history	Stores historical virtual states for temporal analysis and validation.
prediction_results	Stores ML inference outputs, including irrigation need, suggested hour, confidence, action, and input features.
twin_predictions	Stores future moisture projections or time-horizon outputs associated with the Digital Twin.
simulation_scenarios	Stores user-defined scenario inputs such as duration, horizon, and flow-rate override.
simulation_results	Stores outputs of no-irrigation and with-irrigation simulations.
recommendations	Stores generated irrigation recommendations, status, reason, priority, duration, and dose.
decision_rules	Stores rule-based decision parameters or fallback logic configuration.
alerts	Stores warning or critical messages related to sensor health, moisture thresholds, or system state.
irrigation_events	Stores scheduled or executed irrigation events.

Table	Role in the platform
control_commands	Stores commands sent or prepared for actuator execution.
execution_feedback	Stores feedback after pump execution, including status and duration.

I.1 I.1 Why this database supports a Digital Twin

A simple IoT dashboard could store only sensors and readings. Agrio stores much more: virtual states, history, predictions, simulations, recommendations, schedules, commands, and feedback. This structure is a database-level proof that the project was designed as a Digital Twin platform rather than a simple monitoring tool.

The strongest tables for defending the Digital Twin nature of the project are digital_twins, zone_states, zone_state_history, prediction_results, twin_predictions, simulation_scenarios, simulation_results, recommendations, control_commands, irrigation_events, and execution_feedback. Together, these tables connect physical measurement to virtual representation and operational action.

In the defense, the students can explain the database as a chain: sensor_readings and weather_records represent the physical data; zone_states represent the synchronized virtual state; prediction_results and simulation_results represent intelligence and scenario analysis; recommendations and irrigation_events represent decisions; control_commands and execution_feedback represent the link back to the physical actuator.

Appendix J

Detailed API and Service Responsibilities

This appendix summarizes the expected responsibilities of the backend API and internal services. It should be updated if route names change in the final code.

Table J.1: Backend API responsibilities

API group	Responsibility
Users/Farms/Plots/Crops/Soil profiles	CRUD endpoints for master data management.
Zones	Returns zone list, latest state, history, and selected zone details.
Sensors	Registers sensors, maps devices to sensor types, and exposes sensor readings.
Weather	Stores and retrieves weather records and forecast data.
Digital Twins	Returns twin state, synchronization status, and current virtual representation.
Predictions	Stores and exposes ML prediction results.
Simulations	Runs what-if scenarios and returns simulated moisture projections.
Recommendations	Generates, lists, validates, or updates irrigation recommendations.
Schedules/Irrigation events	Creates and manages planned irrigation events.
Device control	Publishes WeMos commands such as ON_D5 and OFF_D5.
Dashboard	Aggregates overview information for frontend cards.
Forecast	Generates weather-aware irrigation outlooks.
System/status/debug	Supports testing, health checks, and implementation verification.

J.1 J.1 Internal services

Table J.2: Internal backend service responsibilities

Service	Responsibility
MQTT parser	Converts incoming JSON payloads into normalized internal sensor/weather fields.
MQTT consumer	Handles the received topic and payload, stores records, and triggers the real-time pipeline.
Mapping service	Resolves device identifiers, zones, and sensor types.
Realtime pipeline	Coordinates sensor health, state update, ML prediction, history, and recommendations.
Digital Twin service	Builds and retrieves current virtual zone states.
Simulator	Computes what-if moisture projections.
ML service client	Sends feature payloads to the Random Forest inference service.
Recommendation engine	Transforms prediction/twin state into recommendations.
Forecast service	Calls weather forecast APIs and generates irrigation outlooks.
Device control service	Publishes MQTT commands to WeMos.
Irrigation scheduler	Executes scheduled irrigation events at the planned time.

J.2 J.2 API evidence to collect

- FastAPI /docs screenshot showing API groups and routes.
- GET /api/v1/dashboard/overview or equivalent response showing dashboard aggregation.
- GET /api/v1/zones/{id}/twin-state or equivalent showing virtual zone state.
- POST simulation endpoint response showing projected moisture and recommendation.
- POST device command endpoint response after ON_D5/OFF_D5 command.

Appendix K

Detailed Algorithms and Pseudocode

This appendix provides implementation-independent pseudocode for the main workflows. These algorithms can be used in Chapter 7 and Chapter 8 if the supervisor prefers a more formal presentation.

K.1 Algorithm K.1 - MQTT sensor ingestion

1. Receive MQTT message with topic and payload.
2. If topic is not sensors/data, route to device-status handler or ignore.
3. Parse payload as JSON.
4. Validate required fields such as device_id and timestamp.
5. Normalize measurement names and units.
6. Resolve management zone from payload or device mapping.
7. Create missing sensor mappings if allowed.
8. Insert weather record when environmental fields exist.
9. Insert one sensor_readings row per normalized measurement.
10. Update runtime MQTT status.
11. Trigger real-time Digital Twin update pipeline.

K.2 Algorithm K.2 - Real-time Digital Twin update

1. Load latest readings and weather records for the zone.
2. Evaluate sensor freshness and health.
3. Compute current moisture, soil temperature, water balance, and stress status.
4. Build ML feature payload.

5. Call ML service /predict.
6. If ML succeeds, store prediction result and update AI fields.
7. If ML fails, compute fallback rule-based decision.
8. Upsert zone__states.
9. Append zone__state__history.
10. Generate or update recommendation.
11. Expose updated state to dashboard.

K.3 Algorithm K.3 - What-if simulation

1. Receive zone__id, irrigation duration, projection horizon, and optional flow rate.
2. Load latest zone state and environment snapshot.
3. Create no-irrigation scenario by applying moisture decrease over horizon.
4. Create with-irrigation scenario by adding irrigation gain then applying evaporation/decay.
5. Select ML path according to recommended action.
6. Compute difference from live state, 24-hour projection, and 48-hour projection.
7. Store simulation scenario and result.
8. Return explanation and curves to frontend.

K.4 Algorithm K.4 - Recommendation validation

1. User opens recommendation preview.
2. User validates recommendation.
3. Backend updates recommendation status to validated.
4. Backend creates control command and execution feedback placeholder if applicable.
5. Backend creates irrigation event for scheduled execution.
6. No direct MQTT command is sent unless user explicitly starts pump or event becomes due.

K.5 Algorithm K.5 - Scheduled irrigation execution

1. Scheduler periodically queries irrigation__events with due start time.
2. For each due event, publish ON_D5 command to WeMos topic.
3. Mark event as running.
4. When end time is reached, publish OFF_D5 command.

5. Mark event as executed.
6. Create execution_feedback record.
7. Update dashboard state.

K.6 K.6 Formula notes for simple moisture projection

The current simulation can be described qualitatively rather than as a final agronomic model. A simple projection combines current moisture, evaporation loss, irrigation gain, and safety bounds. In a future agronomic implementation, this should be replaced or calibrated using a soil water balance model including evapotranspiration, rainfall, infiltration, drainage, field capacity, wilting point, rooting depth, and crop coefficients.

Appendix L

Expanded Chapter Text Blocks for Final Integration

The following text blocks can be moved into the main chapters during final editing if a longer body is required. They are written in thesis style and avoid references to appendices.

L.1 L - Precision agriculture as a systems problem

Precision agriculture should not be reduced to the use of sensors. It is a systems problem that requires the integration of agronomy, electronics, communication, software engineering, data science, and human decision-making. A sensor without a decision workflow only produces data. A machine learning model without reliable data only produces uncertain predictions. A dashboard without actuation remains a monitoring interface. A pump without safety logic can create operational risk. The value of a Digital Twin irrigation system emerges when these components are connected in a coherent cycle that supports reliable decisions.

L.2 L - Why MQTT is appropriate for Agrio

The publish/subscribe model of MQTT is appropriate for Agrio because it decouples the producers and consumers of data. The Raspberry Pi gateway does not need to know which services will consume the data; it only publishes to the sensors/data topic. The backend subscribes to that topic and processes messages. Similarly, the backend does not directly open a physical connection to the WeMos; it publishes commands to the command topic, and the WeMos subscribes. This architecture is flexible because additional subscribers, such as logging services or mobile notification services, can be added without modifying the sensor station.

L.3 L - Why a relational database is useful

A relational database is useful in Agrio because the system contains many linked entities: farms, plots, zones, sensors, readings, weather records, states, predictions, simulations, recommendations, commands, events, and feedback. Relational modeling makes these links explicit through primary keys and foreign keys. It also supports traceability, which is essential for validating decisions. For

example, a recommendation can be linked to the zone state and prediction that produced it, and a command can be linked to the event that triggered it.

L.4 L - Why Random Forest is a pragmatic first model

Random Forest is a pragmatic first model for a final-year smart irrigation prototype because it works well with tabular data, handles nonlinear relations, and does not require the large sequential datasets needed by deep learning models. It is also more robust than a single decision tree because it averages several trees. However, Random Forest is not a substitute for agronomic validation. Its predictions depend on the quality and representativeness of the dataset. Therefore, in Agrio, Random Forest should be interpreted as a decision-support component rather than an absolute agronomic authority.

L.5 L - Why human validation matters

Human validation matters because irrigation decisions have physical consequences. A false positive decision can waste water or cause over-irrigation, while a false negative decision can stress plants. In a prototype, sensor calibration, model uncertainty, and communication reliability are still being evaluated. Keeping the farmer in the loop makes the system safer and more acceptable. It also generates a useful decision history that can later be used to improve the model.

L.6 L - Why the prototype is still valuable without field-scale validation

A controlled prototype cannot prove full agronomic performance, but it can prove system feasibility. It demonstrates that the data chain works, that the backend can process real payloads, that the database supports traceability, that the ML service can be integrated, that the Digital Twin can simulate scenarios, that the dashboard can display decisions, and that the pump can be controlled. These are necessary steps before any real field deployment.

Appendix M

Defense-Oriented Scientific Positioning

This appendix summarizes how the project should be defended in front of the jury. It is not a replacement for the main conclusion, but it provides concise academic arguments.

M.1 M.1 If the jury asks: Why is this a Digital Twin?

- Because the system receives live data from the physical prototype and uses it to update a virtual zone state.
- Because the virtual state is not only displayed; it is used for prediction, simulation, recommendation, and scheduling.
- Because the system can influence the physical prototype through validated or manual pump commands.
- Because it stores history, predictions, simulations, recommendations, commands, and feedback, enabling traceability.

M.2 M.2 If the jury asks: What is the novelty?

- The novelty is not a new sensor or a new ML algorithm alone.
- The novelty is the integration of Libelium/XBee/Raspberry Pi/MQTT/FastAPI/PostgreSQL/ML/Digital Twin/dashboard/WeMos/pump into one coherent prototype.
- The project bridges research state-of-the-art and engineering implementation.
- It provides a human-supervised Digital Twin workflow with simulation before action.

M.3 M.3 If the jury asks: What are the weaknesses?

- The prototype was tested in a controlled environment, not over a full agricultural season.
- ML metrics are dataset-level metrics and need real field validation.

- Security and authentication must be improved for production.
- Remote sensing and advanced agronomic calibration are future work.
- The system currently focuses on one experimental zone.

M.4 M.4 If the jury asks: Why combine Master 2 and Engineering?

- The Master 2 part studies the state of the art and scientific positioning of Digital Twins in agriculture.
- The Engineering part implements and deploys the complete system.
- The combined thesis is justified because Digital Twin projects require both research understanding and engineering realization.